

Neural Networks

Introduction

Agha Ali Raza





The Matrix

Scalars, Vectors and Matrices

(<https://www.mathsisfun.com/algebra/scalar-vector-matrix.html>)

- A scalar is a number e.g., 9, -25, 0.579,...
- A matrix is an array of numbers (one or more rows, one or more columns)

- CS people: Think of multidimensional arrays

- $$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \\ a_{41} & a_{42} & a_{43} \end{bmatrix}$$
 a 4x3 matrix i.e., 4 rows and 3 columns

- $$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$
 $m \times n$ matrix i.e., m rows and n columns

- A vector is a list of numbers (a row or column)

- CS people: Think of a 1D array

- $[25 \quad 4 \quad 3]$ 1x3 vector

- $\begin{bmatrix} 4 \\ -12 \\ 18 \end{bmatrix}$ 3x1 vector

- All vectors are also matrices (as a matrix can be 1D).

- Therefore, the rules that we develop for matrices, also work for vectors

Addition/subtraction

- The two matrices to be added/subtracted must be the same size

$$\begin{bmatrix} 3 & 8 \\ 4 & 6 \end{bmatrix} + \begin{bmatrix} 4 & 0 \\ 1 & -9 \end{bmatrix} = \begin{bmatrix} 7 & 8 \\ 5 & -3 \end{bmatrix}$$

$$\begin{bmatrix} 3 & 8 \\ 4 & 6 \end{bmatrix} - \begin{bmatrix} 4 & 0 \\ 1 & -9 \end{bmatrix} = \begin{bmatrix} -1 & 8 \\ 3 & 15 \end{bmatrix}$$

Negative, Scaler Multiplication, Transpose

$$-\begin{bmatrix} 2 & -4 \\ 7 & 10 \end{bmatrix} = \begin{bmatrix} -2 & 4 \\ -7 & -10 \end{bmatrix}$$

$-(2) = -2$

$$2 \times \begin{bmatrix} 4 & 0 \\ 1 & -9 \end{bmatrix} = \begin{bmatrix} 8 & 0 \\ 2 & -18 \end{bmatrix}$$

$2 \times 4 = 8$

$$\begin{bmatrix} 6 & 4 & 24 \\ 1 & -9 & 8 \end{bmatrix}^T = \begin{bmatrix} 6 & 1 \\ 4 & -9 \\ 24 & 8 \end{bmatrix}$$

Matrix Multiplication

- For two matrices to be compatible for multiplication, the # columns of the first must match the # rows of the second i.e., the inner dimensions must match
- E.g., If A is a matrix with dimensions $m \times n$, and B is a matrix with dimensions $p \times q$, then:
 - We can only produce the product $Y = AB$, if $n = p$
 - The dimensions of the product matrix, Y would be $m \times q$
 - Therefore, $m \times n \times n \times q \rightarrow m \times q$

"Dot Product"

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix} = \begin{bmatrix} 58 & \\ & \end{bmatrix}$$

- The "Dot Product" is where we **multiply matching members**, then sum up:
 - $(1, 2, 3) \cdot (7, 9, 11) = 1 \times 7 + 2 \times 9 + 3 \times 11 = 58$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix} = \begin{bmatrix} 58 & 64 \\ & \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix} = \begin{bmatrix} 58 & 64 \\ 139 & 154 \end{bmatrix}$$

Matrix Multiplication

$$\begin{bmatrix} \$3 & \$4 & \$2 \end{bmatrix} \times \begin{bmatrix} 13 & 9 & 7 & 15 \\ 8 & 7 & 4 & 6 \\ 6 & 4 & 0 & 3 \end{bmatrix} = \begin{bmatrix} \$83 & \$63 & \$37 & \$75 \end{bmatrix}$$

$\$3 \times 13 + \$4 \times 8 + \$2 \times 6$

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} = \begin{bmatrix} 1 \times 4 + 2 \times 5 + 3 \times 6 \end{bmatrix} = \begin{bmatrix} 32 \end{bmatrix}$$

$$\begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} = \begin{bmatrix} 4 \times 1 & 4 \times 2 & 4 \times 3 \\ 5 \times 1 & 5 \times 2 & 5 \times 3 \\ 6 \times 1 & 6 \times 2 & 6 \times 3 \end{bmatrix} = \begin{bmatrix} 4 & 8 & 12 \\ 5 & 10 & 15 \\ 6 & 12 & 18 \end{bmatrix}$$

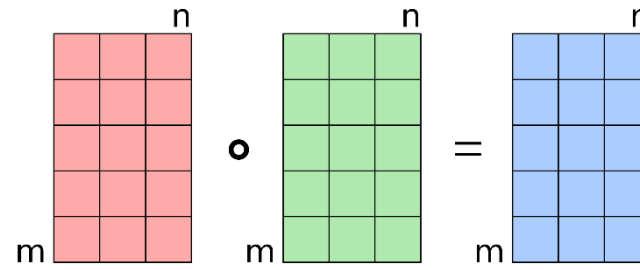
- And by the way, matrix multiplication is not commutative

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 1 \times 2 + 2 \times 1 & 1 \times 0 + 2 \times 2 \\ 3 \times 2 + 4 \times 1 & 3 \times 0 + 4 \times 2 \end{bmatrix} = \begin{bmatrix} 4 & 4 \\ 10 & 8 \end{bmatrix}$$

$$\begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 2 \times 1 + 0 \times 3 & 2 \times 2 + 0 \times 4 \\ 1 \times 1 + 2 \times 3 & 1 \times 2 + 2 \times 4 \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ 7 & 10 \end{bmatrix}$$

Matrix: Hadamard Product

- The Hadamard product (element-wise product) takes two matrices of the **same dimensions** and produces another matrix of the **same dimension** as the operands
 - Each element i, j is the product of elements i, j of the original two matrices.
 - Denoted as $\circ$ or .*


$$A \circ B = C$$

For example, the Hadamard product for a 3×3 matrix A with a 3×3 matrix B is

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \circ \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} = \begin{bmatrix} a_{11} b_{11} & a_{12} b_{12} & a_{13} b_{13} \\ a_{21} b_{21} & a_{22} b_{22} & a_{23} b_{23} \\ a_{31} b_{31} & a_{32} b_{32} & a_{33} b_{33} \end{bmatrix}$$

Sources

- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell, Video Lectures 35-37,
https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecture_note20.html
- **Deep Learning Specialization**, Andrew Ng
https://www.coursera.org/specializations/deep-learning?utm_source=deeplearningai&utm_medium=institutions&utm_campaign=SocialYoutubeDLSC1W1L1
 - Video Lectures: C1W3L1 – C1W4L6:
<https://www.youtube.com/playlist?list=PLpFsSf5Dm-pd5d3rjNtIXUHT-v7bdaEle>
- **A beginner's guide to deriving and implementing backpropagation**, by Pranav Budhwant <https://link.medium.com/Zp3zxNWpf6>
- **Tensorflow playground**: <https://playground.tensorflow.org/>
- **Machine Learning Playground**: <https://ml-playground.com/#>

The Artificial “Neuron”

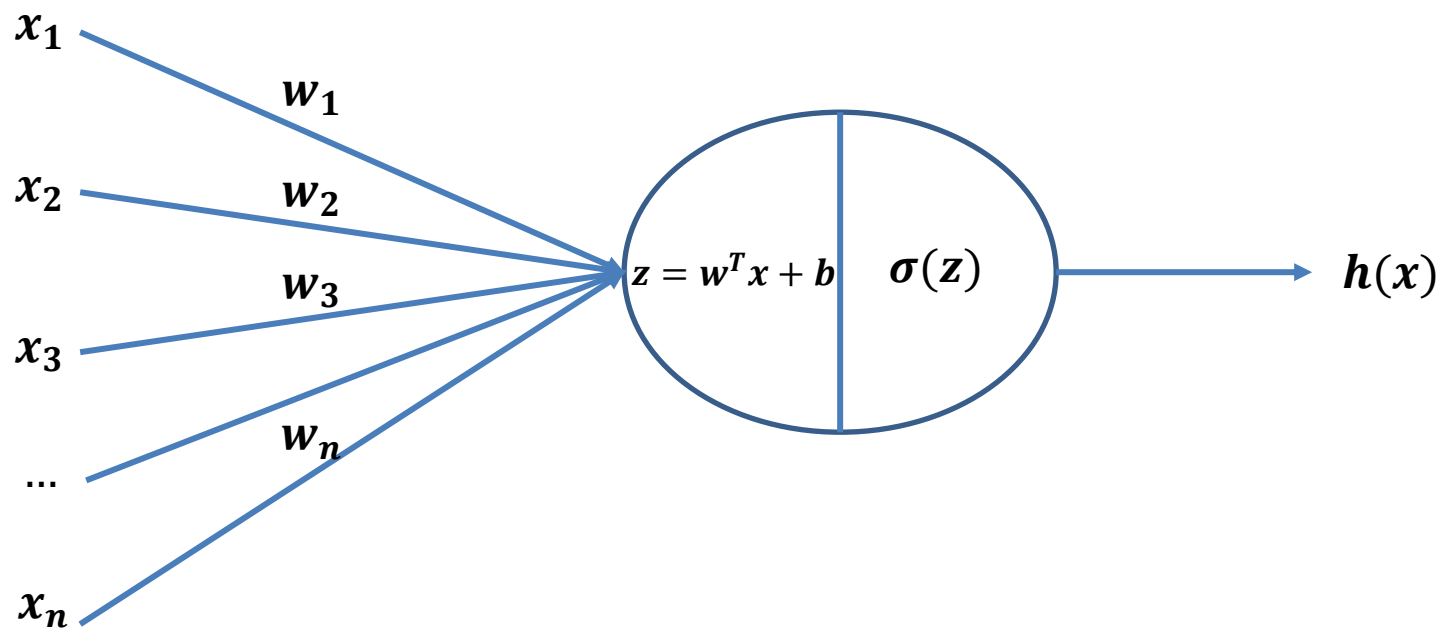
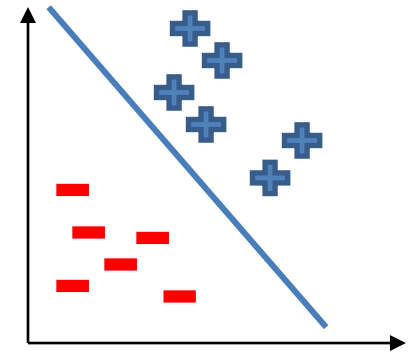
- Remember logistic regression?

$$h(x) = \sigma(w^T x + b)$$

$\Leftrightarrow$

$$z = w^T x + b$$

$$h(x) = \sigma(z)$$



Learning weights

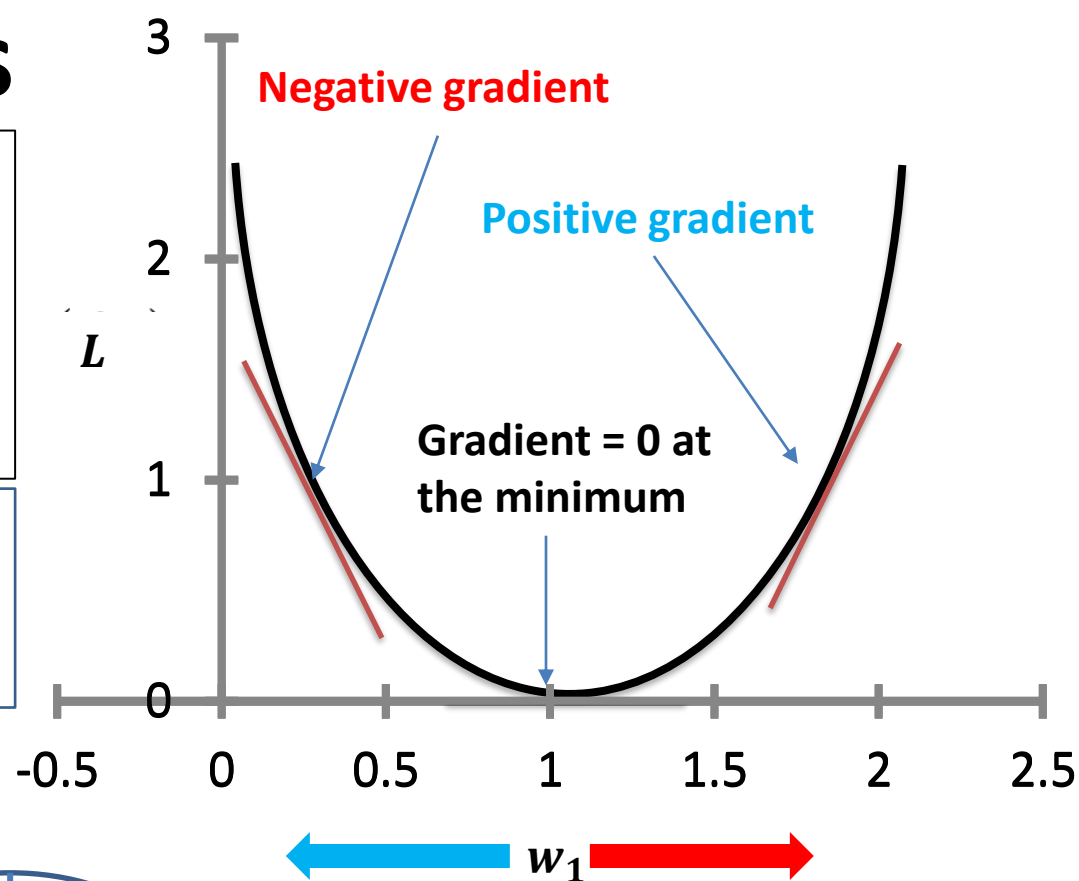
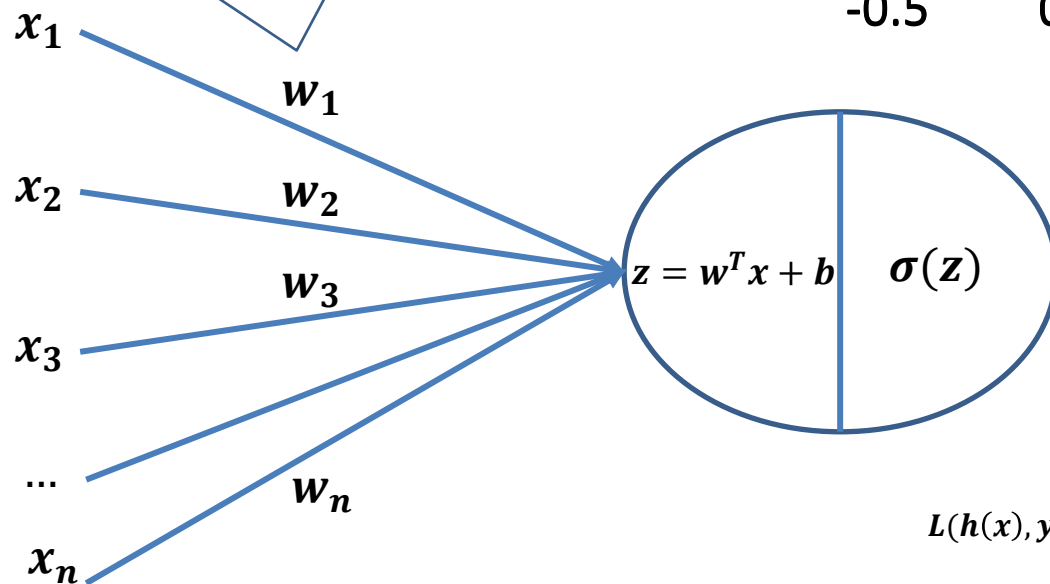
Contribution of w_1 in L : $\frac{\partial}{\partial w_1}(L)$

$$w_1 := w_1 - \alpha \frac{\partial}{\partial w_1}(L)$$

$w_1 := w_1 - \alpha(\text{negative})$: Increases w_1

$w_1 := w_1 - \alpha(\text{positive})$: Decreases w_1

1. What is my contribution in this cost?
2. Should I increase or decrease my value to lower the cost?



A cost function:

$$L(h(x), y) = \frac{1}{2m} \sum_{i=1}^m (h(x)^{(i)} - y^{(i)})^2$$

$$L(h(x), y) = \frac{1}{m} \sum_i -y^{(i)} \log(h(x)^{(i)}) - (1 - y^{(i)}) \log(1 - h(x)^{(i)})$$

Can we do better than linear decision boundaries?

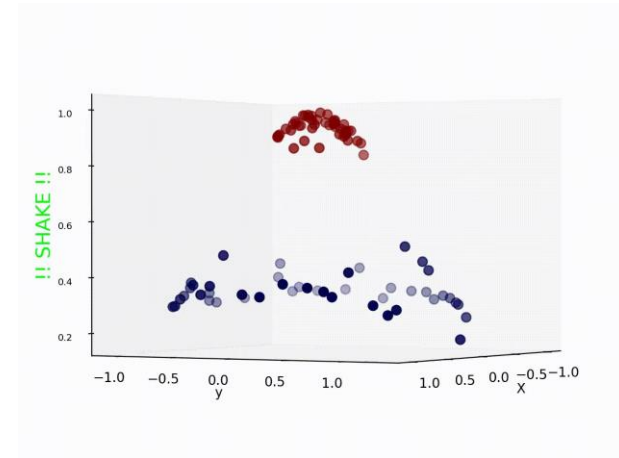
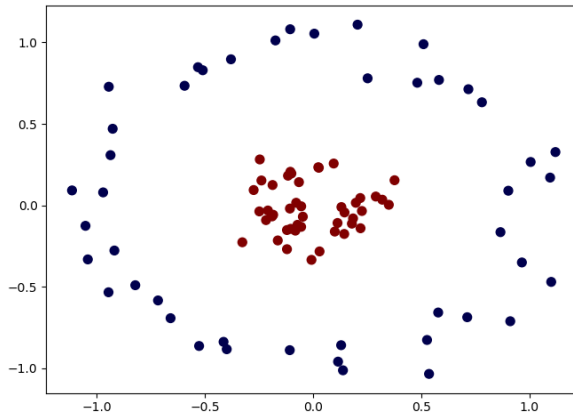
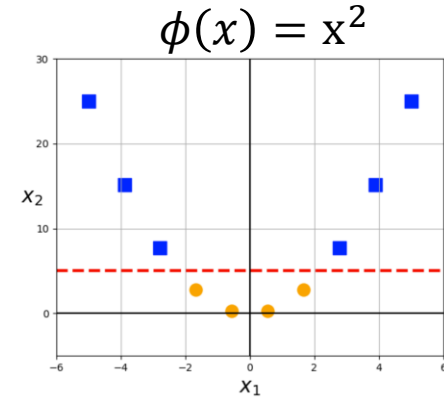
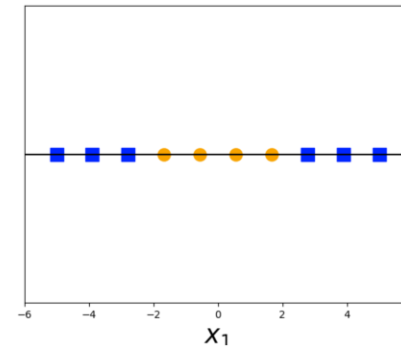
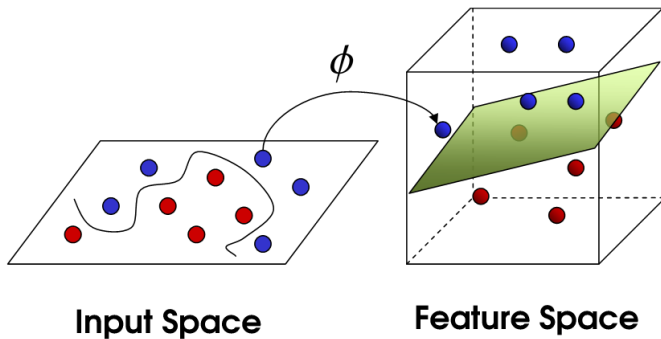
(Image refs: <https://towardsdatascience.com/understanding-support-vector-machine-part-2-kernel-trick-mercers-theorem-e1e6848c6c4d>, <https://towardsdatascience.com/the-kernel-trick-c98cdbcaeb3f>, Weinberger, Lectures 35-37)

$$h(x) = \sigma(w^T x + b)$$

- Manipulate x

$$x \rightarrow \phi(x)$$
$$h(x) = \sigma(w^T \phi(x) + b)$$

- Kernels: Predefined $\phi(x)$.



Neural Networks (Weinberger, Lectures 35-37)

$$h(x) = \sigma(w^T x + b)$$

- Manipulate x
- Neural Networks:
 - Learn $\phi(x)$

$$x \rightarrow \phi(x)$$

$$h(x) = \sigma(w^T \phi(x) + b)$$

- Learn the w , b and the representation of the data $\phi(x)$
- $\phi(x) = g(v^T x + c)$, where $g(z)$ is again a non-linear function like the *Heaviside step function*, *sigmoid*, *hyperbolic tangent (tanh)*, *Rectified Linear Unit (RELU)*, *Leaky RELU*,...

$$h(x) = \sigma(w^T (g(v^T x + c)) + b)$$

- Why non-linear?

$$h(x) = w^T (v^T x + c) + b$$

$$h(x) = w^T v^T x + w^T c + b$$

$$h(x) = (w^T v^T) x + (w^T c + b)$$

- Only a linear function!

Neural Networks (Weinberger, Lectures 35-37)

- With some changes, the NN's have been around for a while
- Multi-layer perceptron → Artificial Neural Network → Deep Learning
- Major changes
 - GPUs – matrix multiplications
 - Preference of activation functions
 - *Stochastic* Gradient Descent (SGD) and *Mini-batch* Gradient Descent
 - Rebranding!



MLP: Who? Perceptron? I am an ANN!

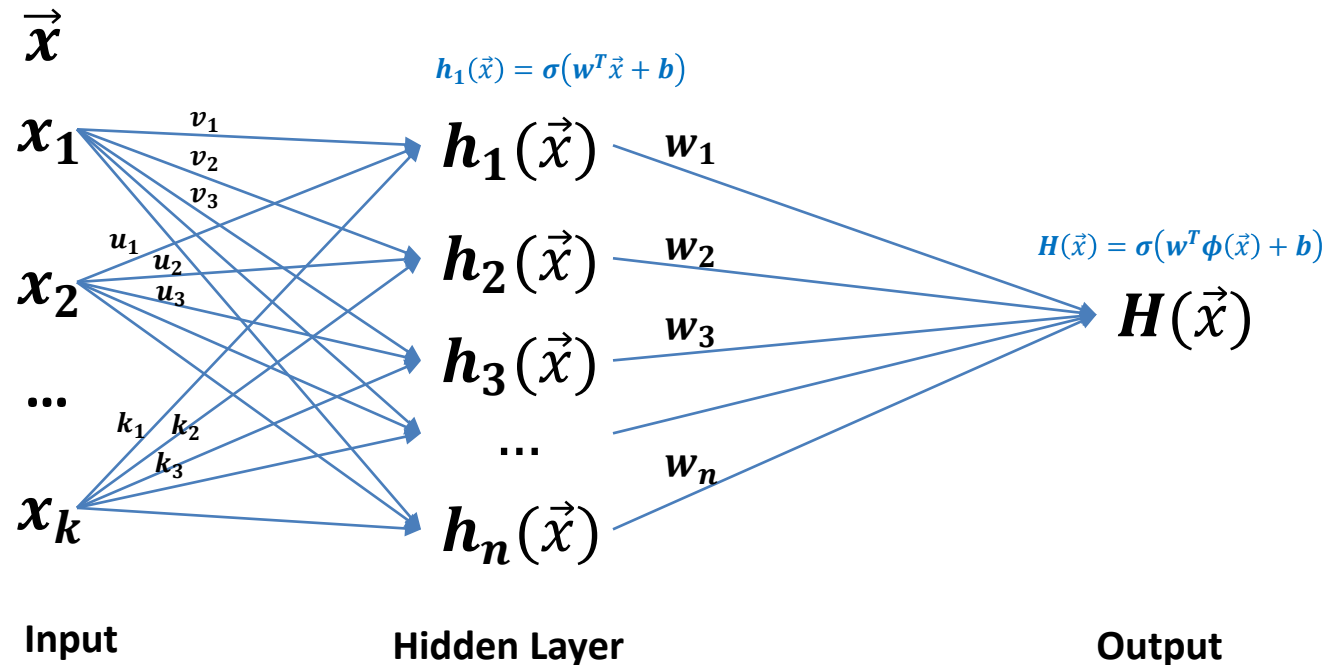
That is **Mr. ANN** for you!

Neural Networks (Weinberger, Lectures 35-37)

$$H(x) = \sigma(w^T \phi(x) + b)$$

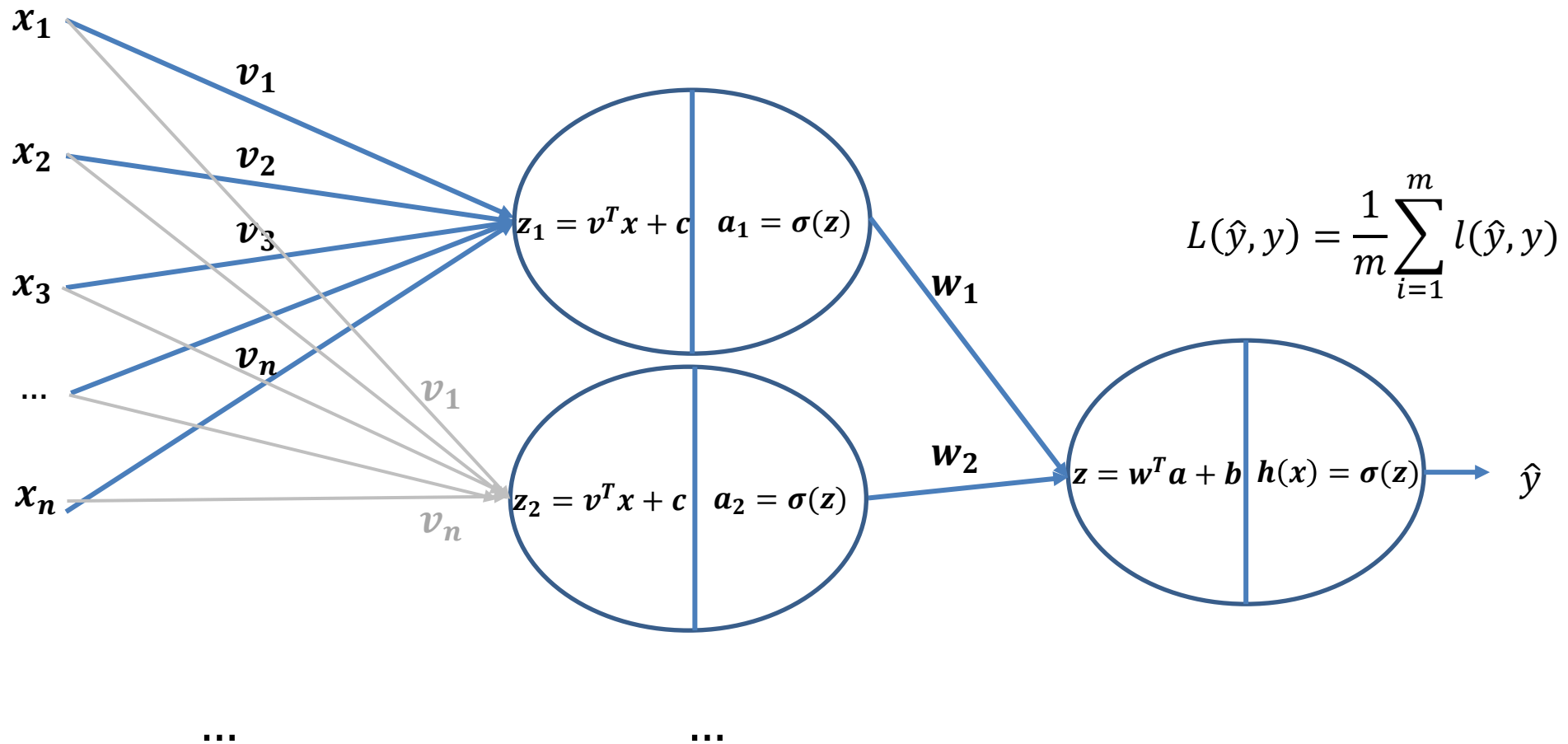
The NN learns $\phi(x)$:

$$\phi(x) = \begin{bmatrix} h_1(x) \\ h_2(x) \\ \dots \\ h_n(x) \end{bmatrix}, \text{ where each } h(x) \text{ is a linear classifier. } h(x) = \sigma(v^T x + c)$$



Neural Networks (Weinberger, Lectures 35-37)

$$h(x) = \sigma(w^T \mathbf{g}(v^T x + c) + b)$$

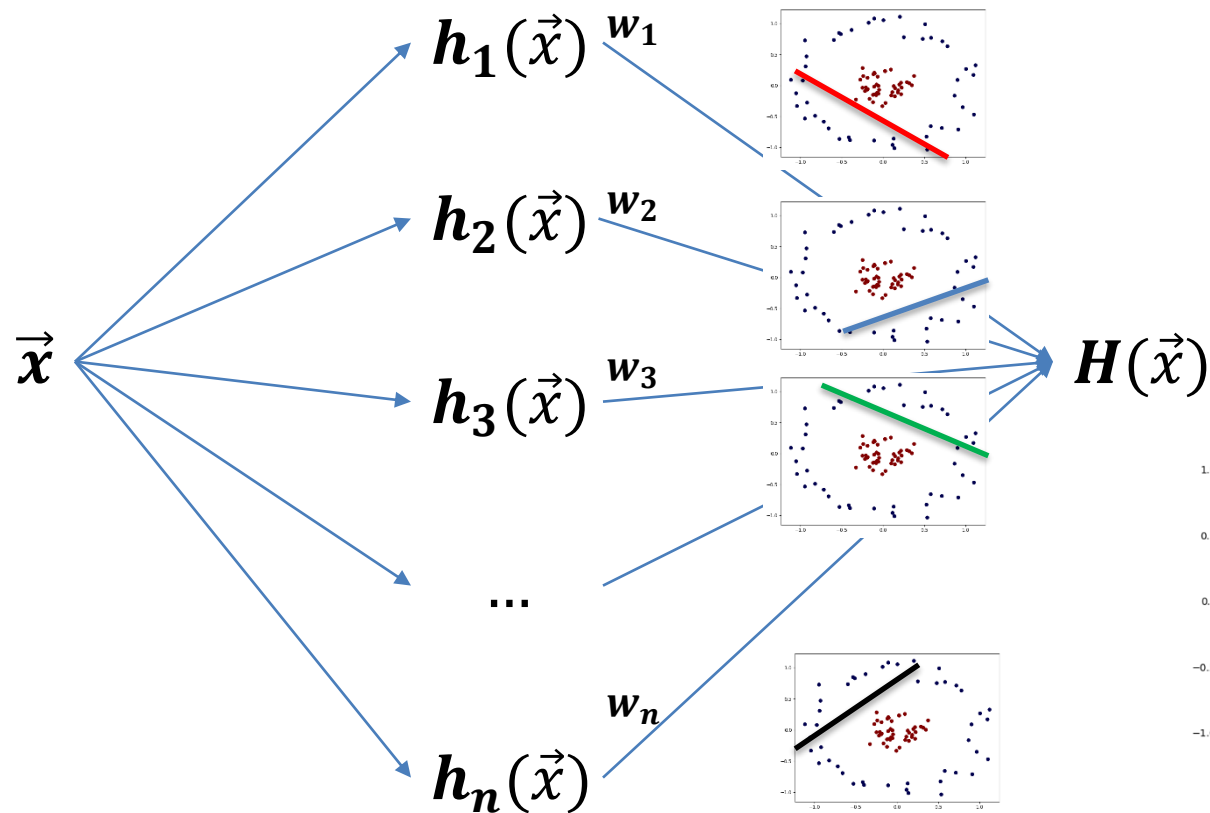


Intuition – How NNs work?

$$H(x) = \sigma(w^T \phi(x) + b)$$

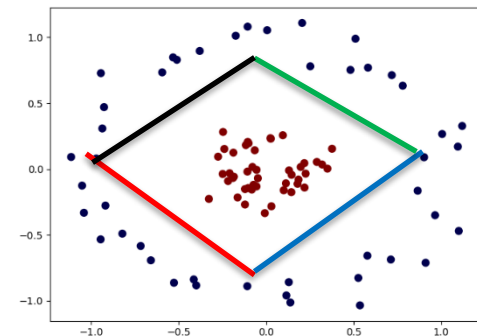
The NN learns $\phi(x)$:

$$\phi(x) = \begin{bmatrix} h_1(x) \\ h_2(x) \\ \dots \\ h_n(x) \end{bmatrix}, \text{ where each } h(x) \text{ is a linear classifier. } h(x) = v^T x + c$$



Becomes smoother
with higher n !

Universal
approximators.



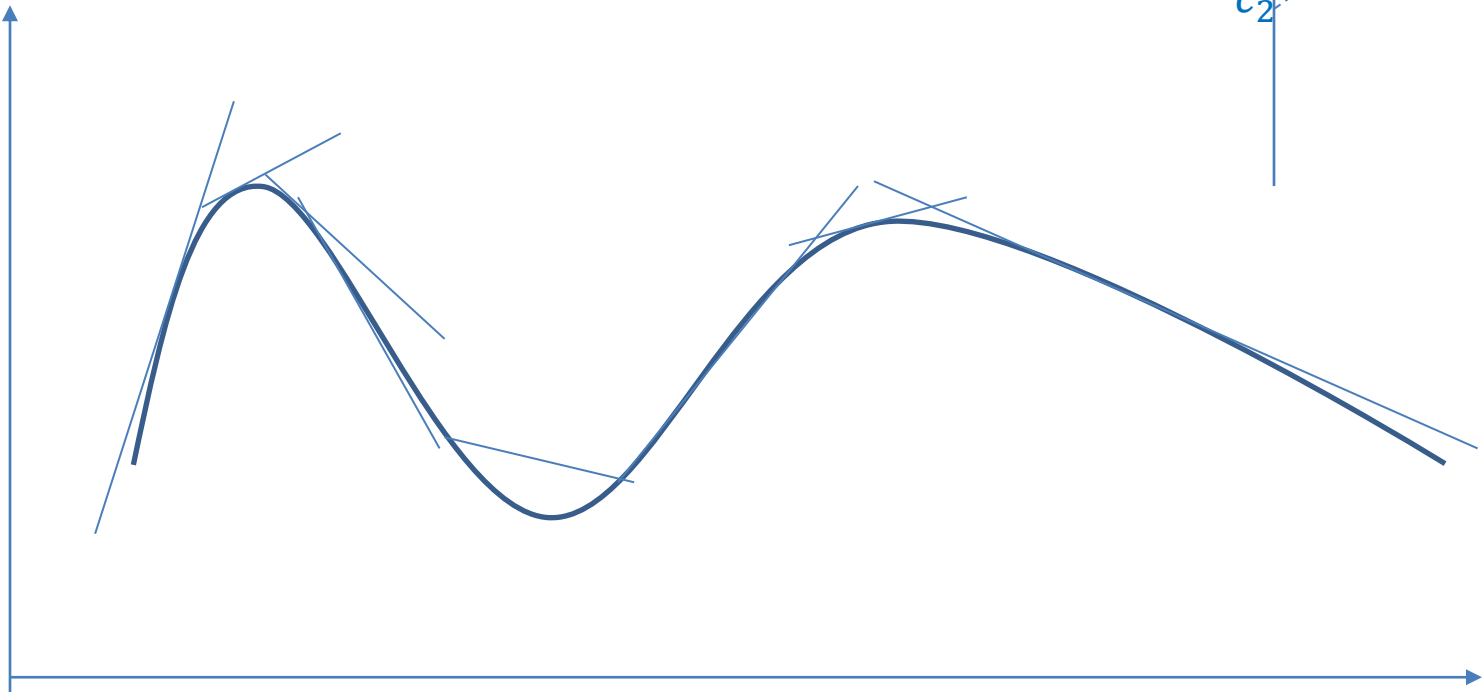
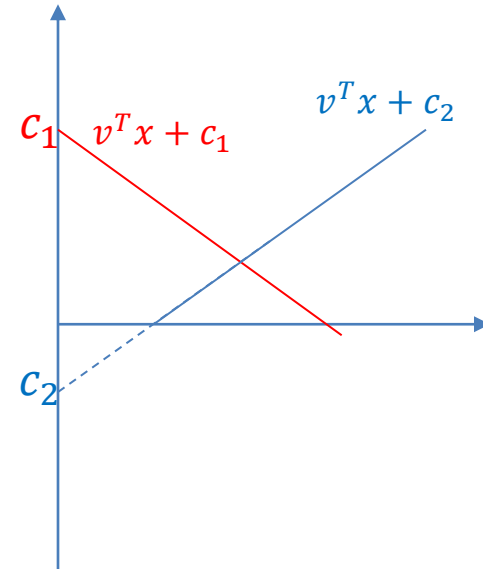
More Intuition – Regression (Weinberger, Lectures 35-37)

RELU

$$H(x) = w^T \phi(x) + b$$

$$H(x) = w^T \max(v^T x + c, 0) + b$$

$$L(H(x), y) = \frac{1}{2m} \sum_{i=1}^m (h(x_i) - y_i)^2$$



Layers (Weinberger, Lectures 35-37)

$$h(x) = \sigma(w^T \phi(x) + b)$$
$$\phi(x) = \sigma(v^T x + c)$$



- To make this more powerful:
 - Either make the matrix v really big
 - Or add layers like a Matryoshka doll

$$h(x) = \sigma(w^T \phi(x) + b)$$
$$\phi(x) = \sigma(v^T \phi'(x) + c)$$
$$\phi'(x) = \sigma(v'^T \phi''(x) + c')$$
$$\phi''(x) = \sigma(v''^T x + c'')$$

- “Deep” learning
 - This was known since the time of Frank Rosenblatt
 - These were not used because:
 - These used to take a long time to process
 - Any function that you can learn with a deep network, you can also learn with a shallow network
 - But in practice, a shallow network requires an exponentially wide matrix to be able to compete with a deep network that needs fewer, smaller matrices to do the same
 - **A deep network allows to learn simple things (lines, hyperplanes, piece-wise linear functions) with earlier layers and build with more complex tools (non-linear functions) in the later layers**

Learning with Layers (Weinberger, Lectures 35-37)

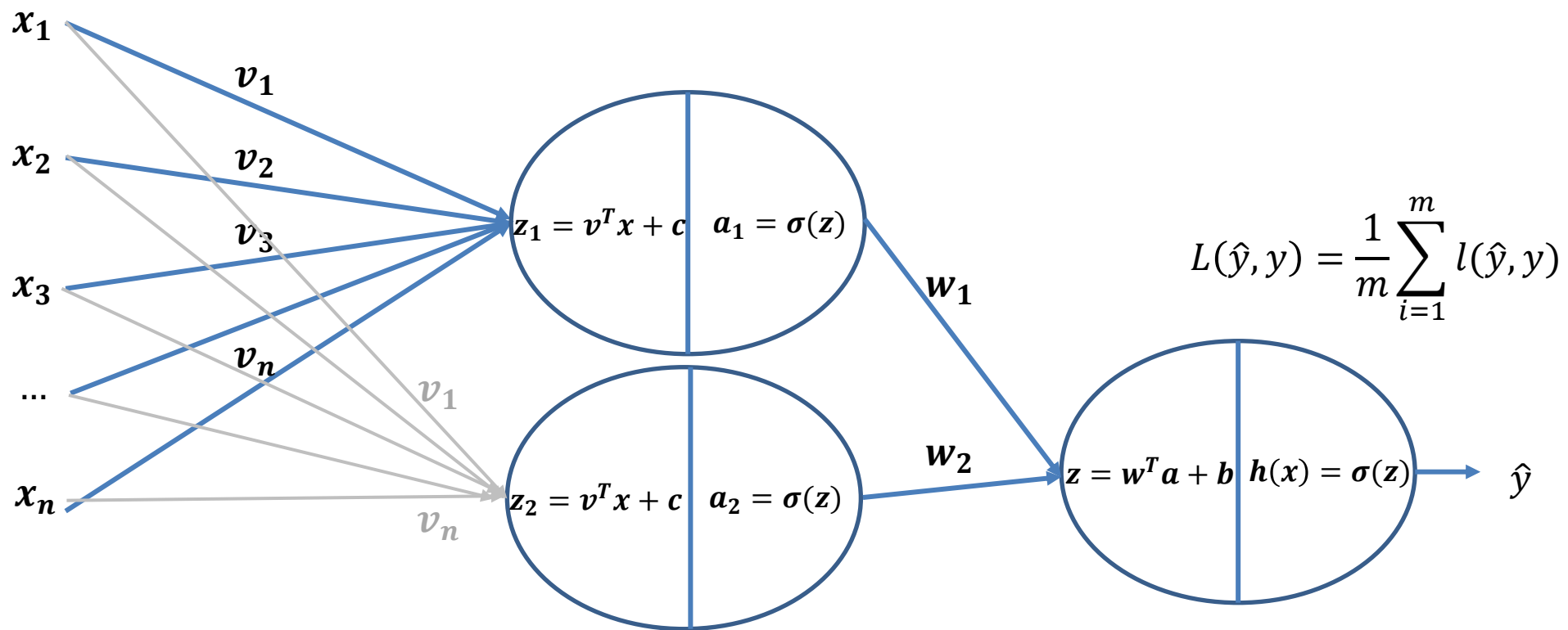
$$h(x) = w^T \phi(x)$$

$$\phi(x) = \sigma(a(x)), a(x) = v^T \phi'(x)$$

$$\phi'(x) = \sigma(a'(x)), a'(x) = v'^T \phi''(x)$$

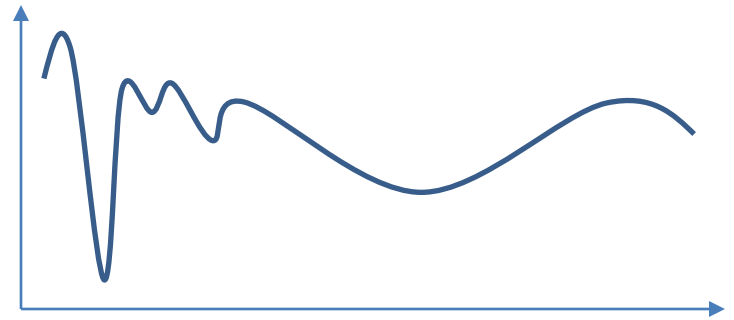
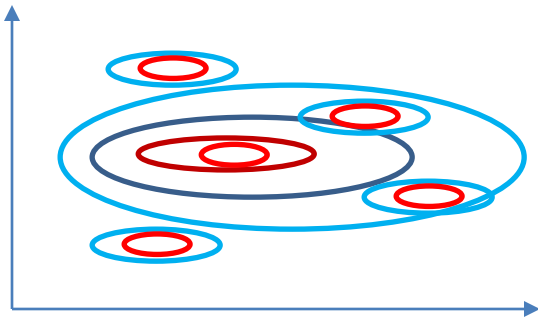
$$\phi''(x) = \sigma(a''(x)), a''(x) = v''^T x$$

- Forward pass and backpropagation



Neural Networks - SGD (Weinberger, Lectures 35-37)

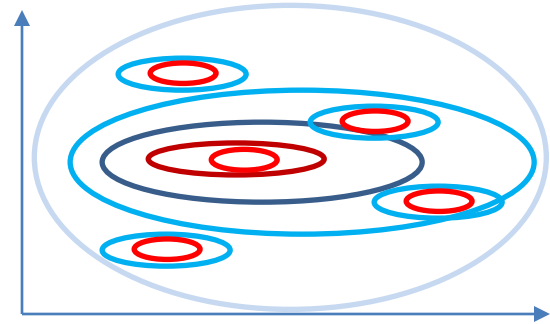
- Forget convex cost functions!
- Approximate the gradient of the cost function with one data point – in random order
 - Compared to batch gradient descent that averages over the whole dataset
- Clearly a bad approximation – and that's why we use it



- It's a bad approximation to find the exact minimum – which we no longer care about
- Misses narrow (may be deep) holes and converges onto wide holes
 - The narrow holes may not be in the same place in the test data
- Does not overfit to data easily
- Is faster as SGD has already taken m steps after one epoch
- The direction of the small steps remains correct **on average**

Neural Networks - SGD (Weinberger, Lectures 35-37)

- In practice, we use the **mini-batch gradient descent**
- Initially use a very large learning rate – prevent from falling into the narrow local minima
- Now you are jumping around in the wide holes
- Lower the learning rate – say by a factor of 10
- And the gradient descent will converge further
- We must remember, that we have billions of local minima
 - Millions of wider holes
- In practice reaching a descent minimum allows for a good enough error rate
 - a NN with a different local minimum may perform equally well
 - So, ensemble methods (that combine classifiers) work well



Up next

- Formalize the notation
 - Logistic regression – Vectorized version
 - NN – Vectorized version
 - Forward pass
- Activation functions
 - Sigmoid, tanh, ReLU, leaky ReLU
 - Pros and cons
 - Gradients
- Forward and Backward passes
 - Backpropagation
 - Logistic Regression
 - NN

Neural Networks

Notation and Forward Pass

Agha Ali Raza

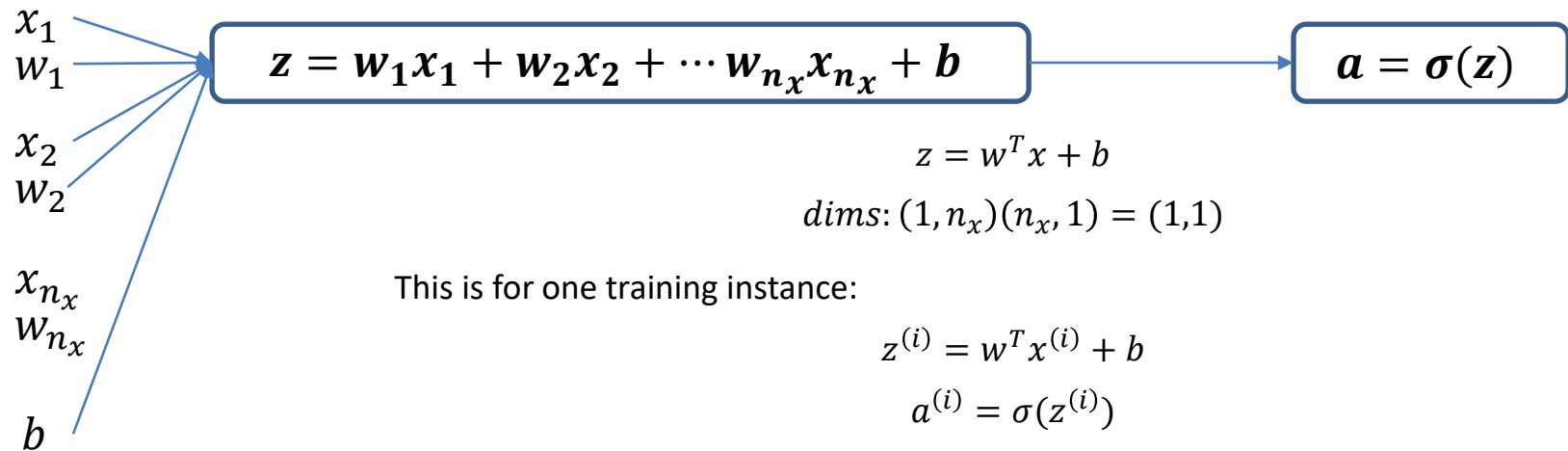


Sources

- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell, Video Lectures 35-37,
https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecture_note20.html
- **Deep Learning Specialization**, Andrew Ng
https://www.coursera.org/specializations/deep-learning?utm_source=deeplearningai&utm_medium=institutions&utm_campaign=SocialYoutubeDLSC1W1L1
 - Video Lectures: C1W3L1 – C1W4L6:
<https://www.youtube.com/playlist?list=PLpFsSf5Dm-pd5d3rjNtIXUHT-v7bdaEle>
- **A beginner's guide to deriving and implementing backpropagation**, by Pranav Budhwant <https://link.medium.com/Zp3zxNWpf6>
- **Tensorflow playground**: <https://playground.tensorflow.org/>
- **Machine Learning Playground**: <https://ml-playground.com/#>

Vectorizing Logistic Regression – The Forward Pass

(Deep Learning Specialization, Andrew Ng)



$$z = w^T x + b$$

$$\text{dims: } (1, n_x)(n_x, 1) = (1, 1)$$

This is for one training instance:

$$z^{(i)} = w^T x^{(i)} + b$$

$$a^{(i)} = \sigma(z^{(i)})$$

We can do better:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_{n_x} \end{bmatrix}, w = \begin{bmatrix} w_1 \\ w_2 \\ \dots \\ w_{n_x} \end{bmatrix}, b = b$$

$$X = \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & x_1^{(3)} & \dots & x_1^{(m)} \\ x_2^{(1)} & x_2^{(2)} & x_2^{(3)} & \dots & x_2^{(m)} \\ \dots & \dots & \dots & \dots & \dots \\ x_{n_x}^{(1)} & x_{n_x}^{(2)} & x_{n_x}^{(3)} & \dots & x_{n_x}^{(m)} \end{bmatrix}, \text{dims} = (n_x, m)$$

$$\mathbf{Z} = \mathbf{w}^T \mathbf{X} + \mathbf{b}$$

$$\text{dims: } (1, m) = (1, n_x)(n_x, m) + (1, m)$$

Note: b would be implicitly converted to $[b \ b \ b \dots b]$ by python, NumPy broadcasting operation.

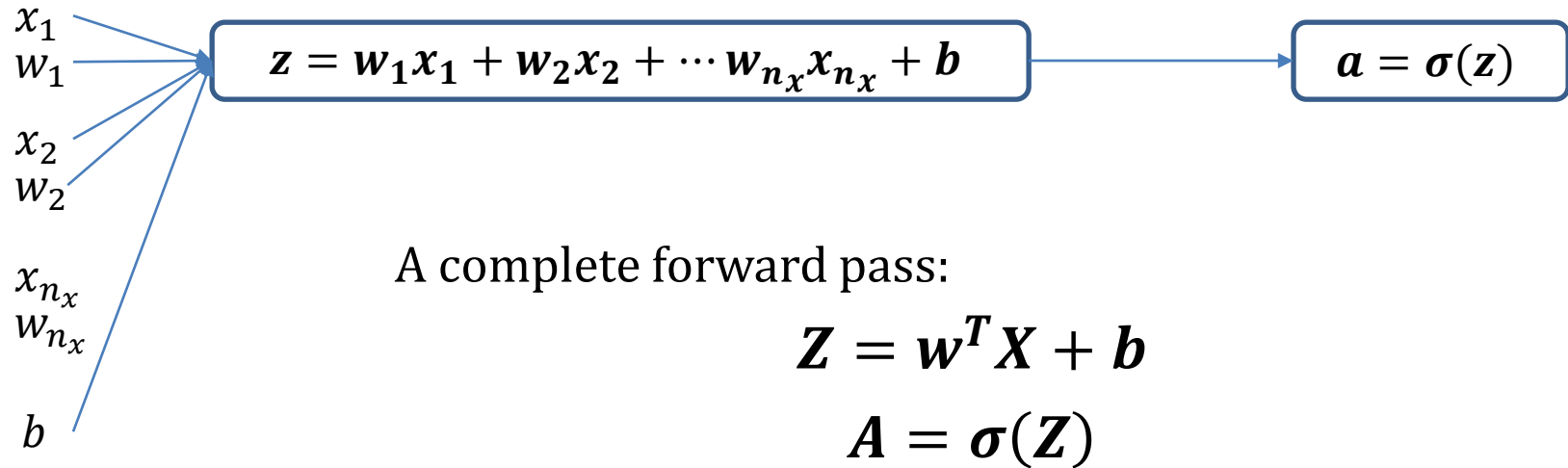
$$\mathbf{Z} = [z^{(1)} \ z^{(2)} \ z^{(3)} \ \dots \ z^{(m)}]$$

$$\mathbf{A} = \sigma(\mathbf{Z})$$

$$\mathbf{A} = [a^{(1)} a^{(2)} a^{(3)} \dots a^{(m)}]$$

Vectorizing Logistic Regression – The Forward Pass

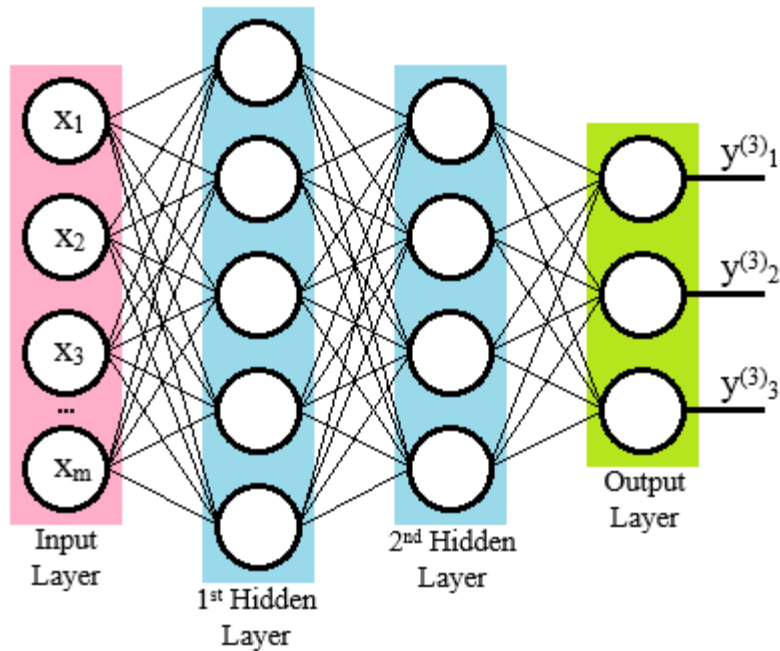
(Deep Learning Specialization, Andrew Ng)



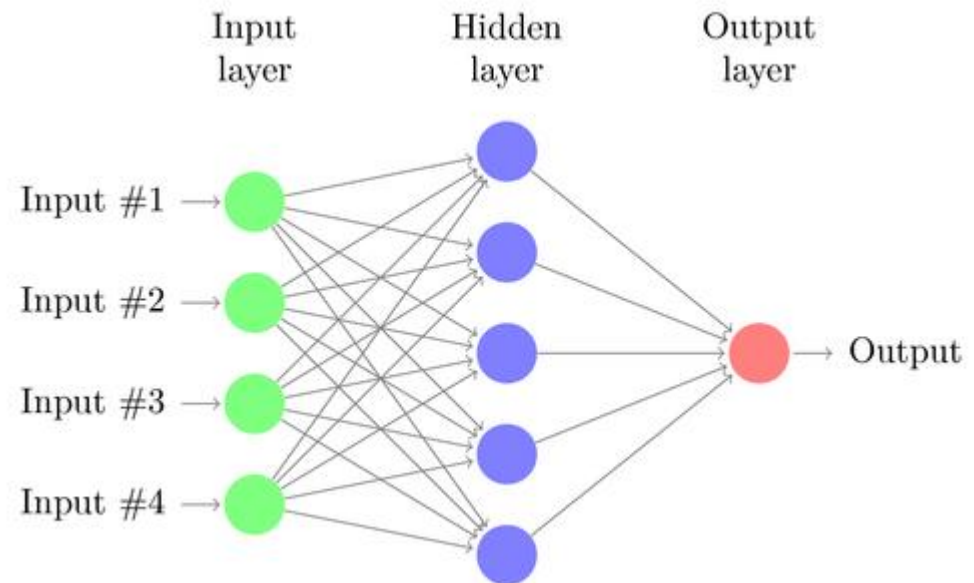
Neural Networks: Terminology

(<https://www.codeproject.com/Articles/1261763/ANNT-Feed-forward-fully-connected-neural-networks>,
<https://laptrinhx.com/deep-learning-using-keras-3760648021/>)

A 3-layered fully-connected network



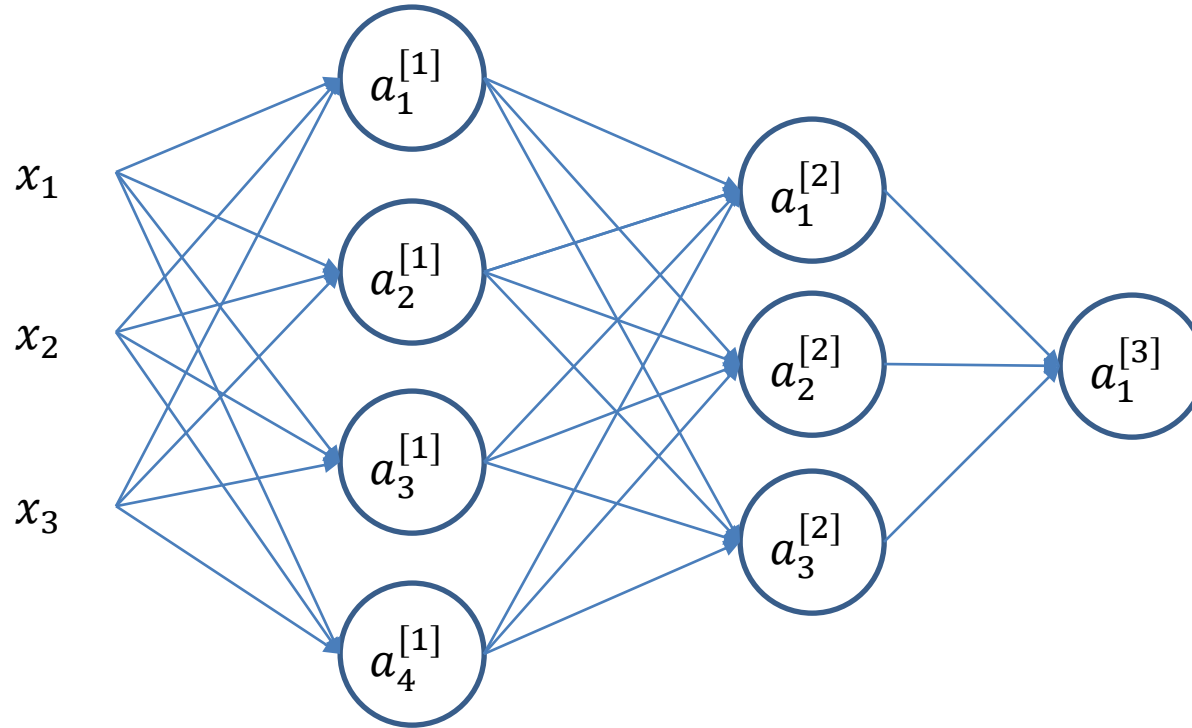
A 2-layered fully-connected network



Formalizing Notation – NN (Deep Learning Specialization, Andrew Ng)

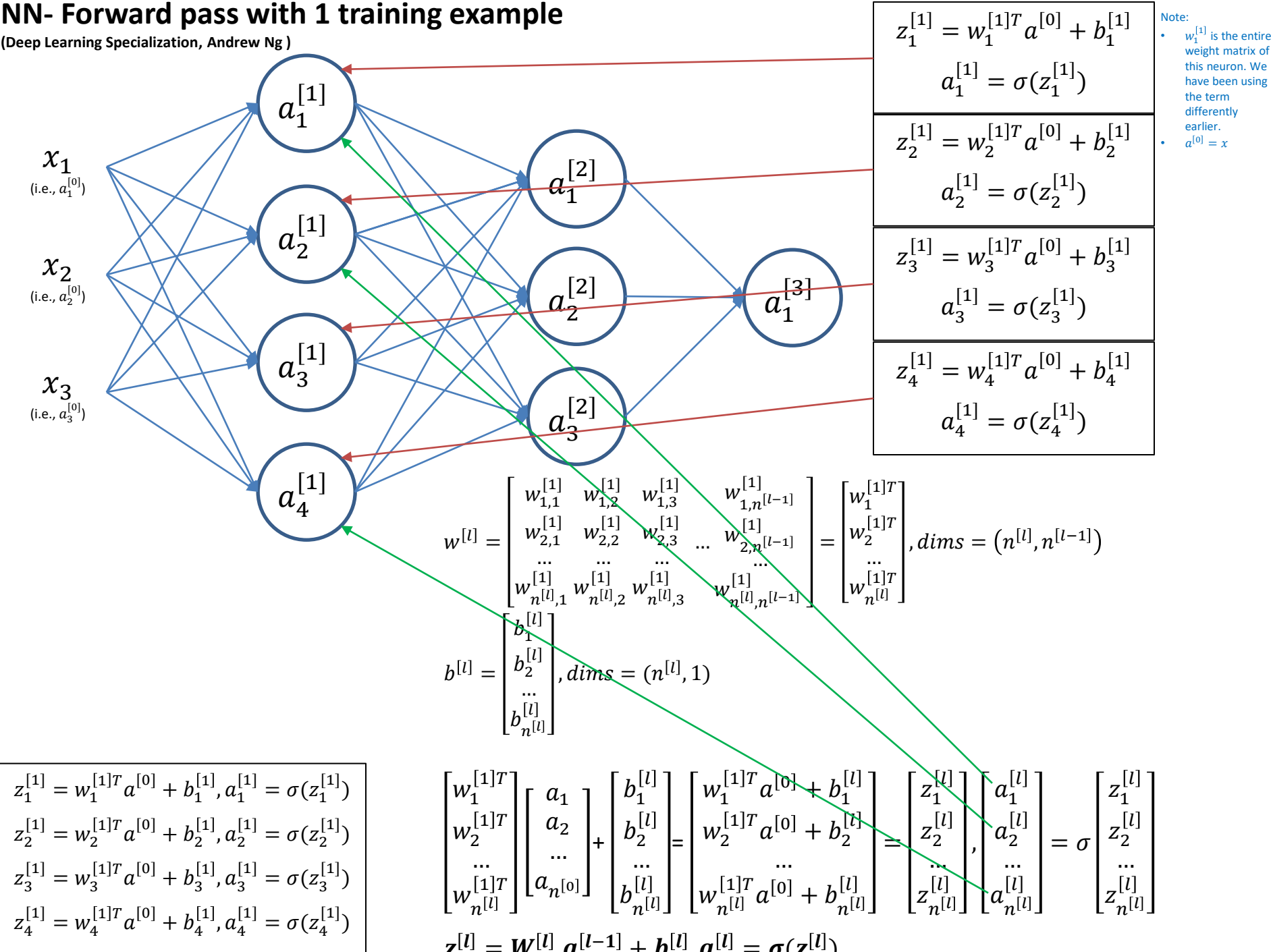
Consider the following 3-layer NN:

- Number of nodes in layer l : $n^{[l]}$
- $a_i^{[l]}$: i th node in layer l
- $a^{[0]} = x$
- $a^{[L]} = h(x) = \hat{y}$



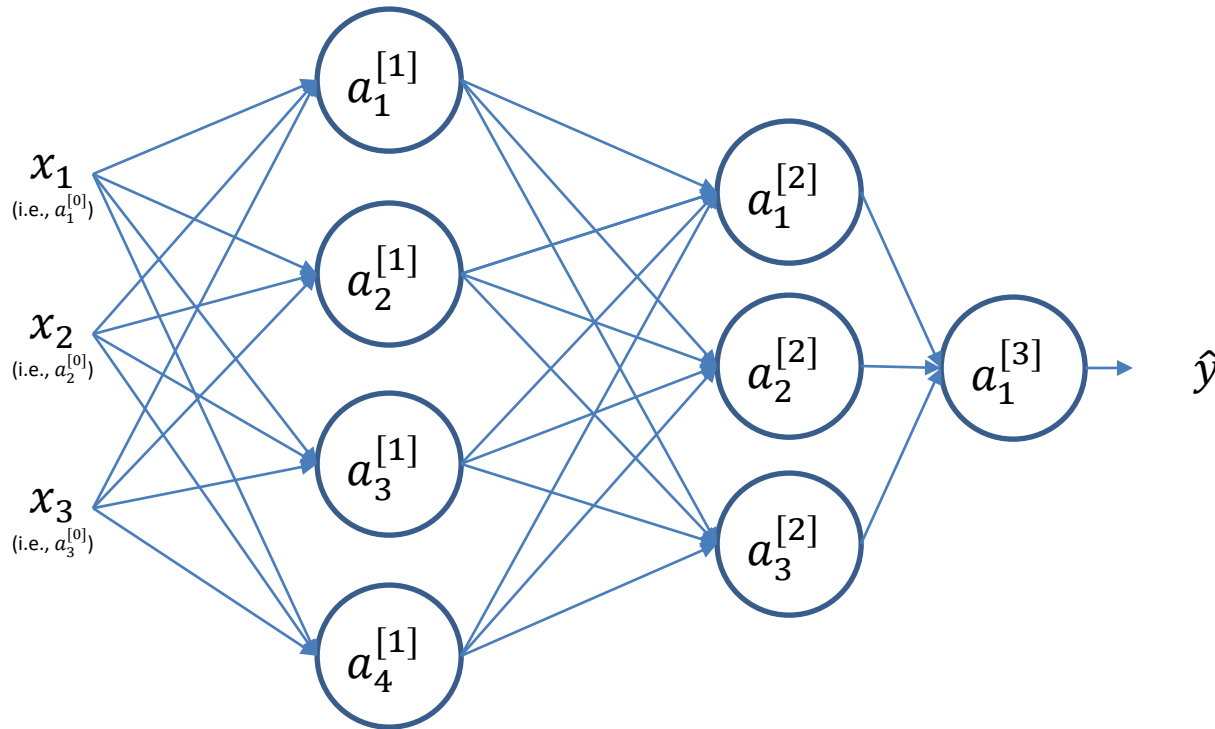
NN- Forward pass with 1 training example

(Deep Learning Specialization, Andrew Ng)



NN- Forward pass with 1 training example

(Deep Learning Specialization, Andrew Ng)



Forward pass

$$\mathbf{z}^{[1]} = \mathbf{W}^{[1]} \mathbf{a}^{[0]} + \mathbf{b}^{[1]}$$

$$(4,1) = (4,3)(3,1) + (4,1)$$

$$\mathbf{a}^{[1]} = \sigma(\mathbf{z}^{[1]})$$

$$(4,1) = (4,1)$$

$$\mathbf{z}^{[2]} = \mathbf{W}^{[2]} \mathbf{a}^{[1]} + \mathbf{b}^{[2]}$$

$$(3,1) = (3,4)(4,1) + (3,1)$$

$$\mathbf{a}^{[2]} = \sigma(\mathbf{z}^{[2]})$$

$$(3,1) = (3,1)$$

$$\mathbf{z}^{[3]} = \mathbf{W}^{[3]} \mathbf{a}^{[2]} + \mathbf{b}^{[3]}$$

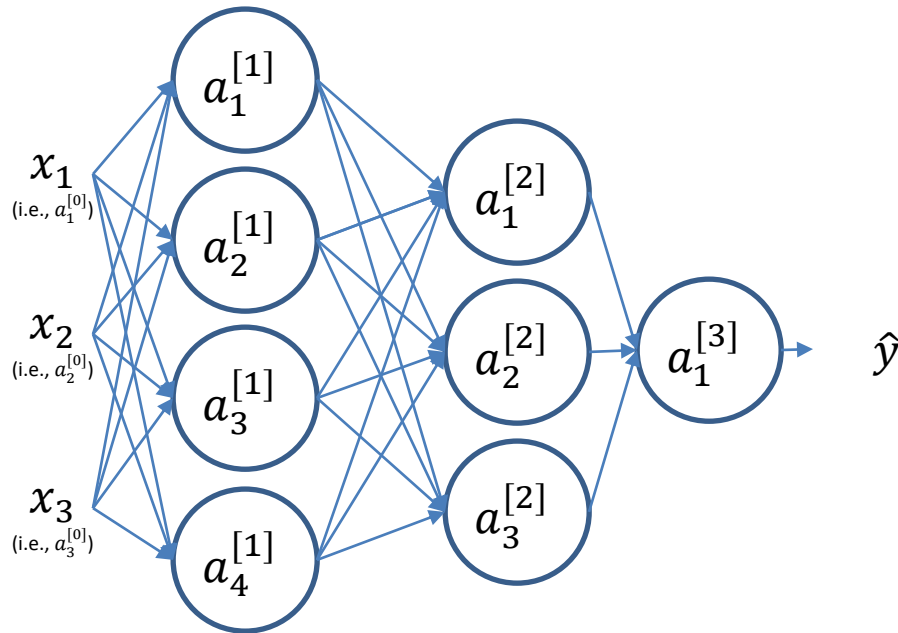
$$(1,1) = (1,3)(3,1) + (1,1)$$

$$\mathbf{a}^{[3]} = \sigma(\mathbf{z}^{[3]})$$

$$(1,1) = (1,1)$$

NN- Forward pass with m training examples

(Deep Learning Specialization, Andrew Ng)



$$X^{(1)} \rightarrow \hat{y}^{(1)} = a^{[3]}(1)$$

$$X^{(2)} \rightarrow \hat{y}^{(2)} = a^{[3]}(2)$$

$$X^{(m)} \rightarrow \hat{y}^{(m)} = a^{[3]}(m)$$

Forward pass with i th instance

$$z^{[1]}(i) = W^{[1]}(i) a^{[0]}(i) + b^{[1]}(i)$$

$$(4,1) = (4,3)(3,1) + (4,1)$$

$$a^{[1]}(i) = \sigma(z^{[1]}(i))$$

$$(4,1) = (4,1)$$

$$z^{[2]}(i) = W^{[2]}(i) a^{[1]}(i) + b^{[2]}(i)$$

$$(3,1) = (3,4)(4,1) + (3,1)$$

$$a^{[2]}(i) = \sigma(z^{[2]}(i))$$

$$(3,1) = (3,1)$$

$$z^{[3]}(i) = W^{[3]}(i) a^{[2]}(i) + b^{[3]}(i)$$

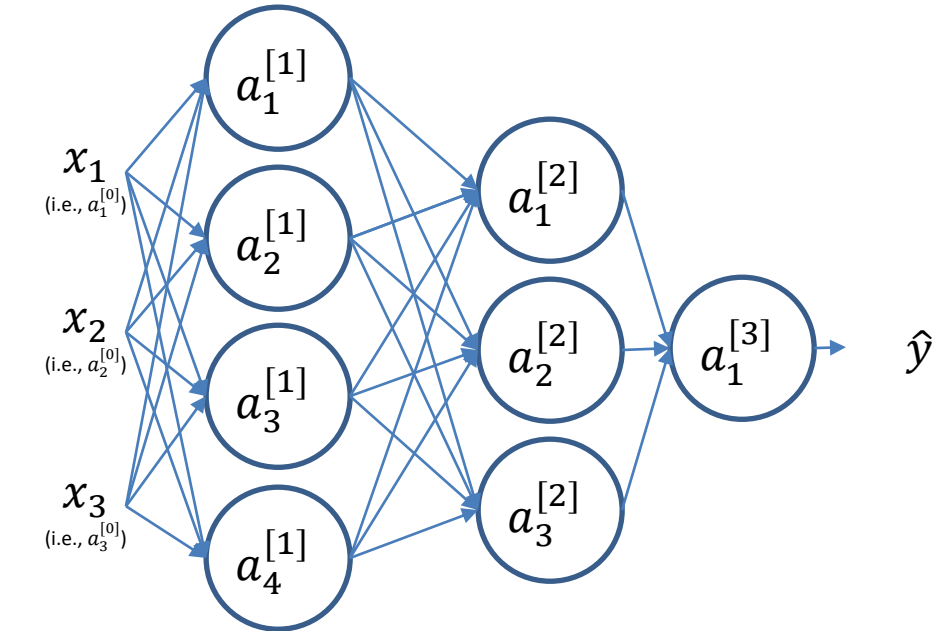
$$(1,1) = (1,3)(3,1) + (1,1)$$

$$a^{[3]}(i) = \sigma(z^{[3]}(i))$$

$$(1,1) = (1,1)$$

NN- Forward pass with m training examples

(Deep Learning Specialization, Andrew Ng)



Using:

$$X = A^{[0]} = \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & x_1^{(3)} & \dots & x_1^{(m)} \\ x_2^{(1)} & x_2^{(2)} & x_2^{(3)} & \dots & x_2^{(m)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_{n^{[0]}}^{(1)} & x_{n^{[0]}}^{(2)} & x_{n^{[0]}}^{(3)} & \dots & x_{n^{[0]}}^{(m)} \end{bmatrix}, \text{dims} = (n^{[0]}, m)$$

$$Z^{[l]} = [z^{[l](1)} \quad z^{[l](2)} \quad z^{[l](3)} \quad \dots \quad z^{[l](m)}]$$
$$Z^{[l]} = \begin{bmatrix} z_1^{[l](1)} & z_1^{[l](2)} & z_1^{[l](m)} \\ z_2^{[l](1)} & z_2^{[l](2)} & z_2^{[l](m)} \\ \vdots & \vdots & \vdots \\ z_{n^{[l]}}^{[l](1)} & z_{n^{[l]}}^{[l](2)} & z_{n^{[l]}}^{[l](m)} \end{bmatrix}$$
$$A^{[l]} = [a^{[l](1)} \quad a^{[l](2)} \quad a^{[l](3)} \quad \dots \quad a^{[l](m)}]$$
$$A^{[l]} = \begin{bmatrix} a_1^{[l](1)} & a_1^{[l](2)} & a_1^{[l](m)} \\ a_2^{[l](1)} & a_2^{[l](2)} & a_2^{[l](m)} \\ \vdots & \vdots & \vdots \\ a_{n^{[l]}}^{[l](1)} & a_{n^{[l]}}^{[l](2)} & a_{n^{[l]}}^{[l](m)} \end{bmatrix}$$

Forward pass with m instances

$$Z^{[1]} = W^{[1]} A^{[0]} + b^{[1]}$$

(4,m) = (4,3)(3,m) + (4,m)

$$A^{[1]} = \sigma(Z^{[1]})$$

(4,m) = (4,m)

$$Z^{[2]} = W^{[2]} A^{[1]} + b^{[2]}$$

(3,m) = (3,4)(4,m) + (3,m)

$$A^{[2]} = \sigma(Z^{[2]})$$

(3,m) = (3,m)

$$Z^{[3]} = W^{[3]} A^{[2]} + b^{[3]}$$

(1,m) = (1,3)(3,m) + (1,m)

$$A^{[3]} = \sigma(Z^{[3]})$$

(1,m) = (1,m)

Up next

- Activation functions
 - Sigmoid, tanh, ReLU, leaky ReLU
 - Pros and cons
 - Gradients
- Forward and Backward passes
 - Backpropagation
 - Logistic Regression
 - NN

Neural Networks

Activation Functions

Agha Ali Raza



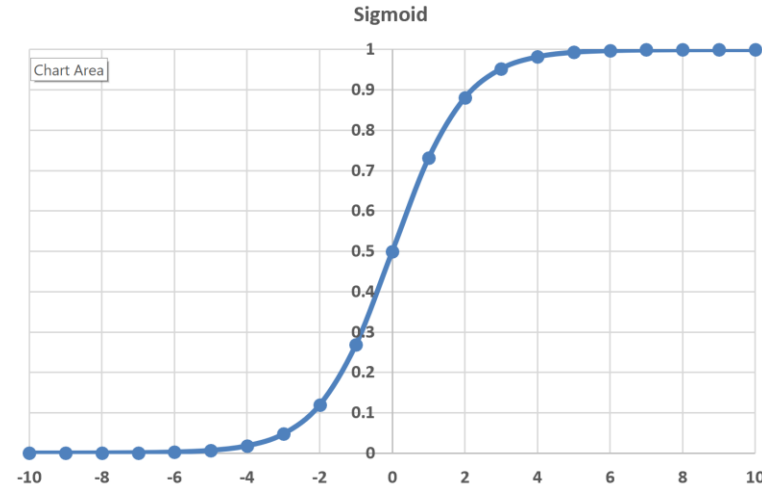
Sources

- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell, Video Lectures 35-37,
https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecture_note20.html
- **Deep Learning Specialization**, Andrew Ng
https://www.coursera.org/specializations/deep-learning?utm_source=deeplearningai&utm_medium=institutions&utm_campaign=SocialYoutubeDLSC1W1L1
 - Video Lectures: C1W3L1 – C1W4L6:
<https://www.youtube.com/playlist?list=PLpFsSf5Dm-pd5d3rjNtIXUHT-v7bdaEle>
- **A beginner's guide to deriving and implementing backpropagation**, by Pranav Budhwant <https://link.medium.com/Zp3zxNWpf6>
- **Tensorflow playground**: <https://playground.tensorflow.org/>
- **Machine Learning Playground**: <https://ml-playground.com/#>

Sigmoid (σ) (Deep Learning Specialization, Andrew Ng)

$$g(z) = \frac{1}{1 + e^{-z}}$$

- Range: $(0, +1)$
- Gradient $\rightarrow 0$ for higher values of $|z|$
- Mean around 0.5
- **Derivative:**



$$\frac{d}{dz} g(z) = g'(z) = \frac{1}{1 + e^{-z}} \left(1 - \frac{1}{1 + e^{-z}} \right)$$
$$g'(z) = g(z)(1 - g(z))$$

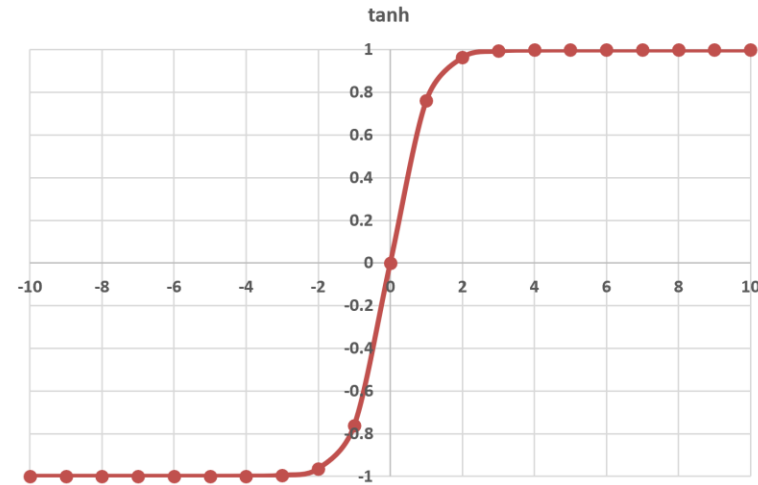
$$\text{if } a = g(z), g'(z) = a(1 - a)$$

- Examples:
 - $Z = 10, g(z) \approx 1, g'(z) \approx 0$
 - $Z = -10, g(z) \approx 0, g'(z) \approx 0$
 - $Z = 0, g(z) \approx \frac{1}{2}, g'(z) = \frac{1}{2} \left(1 - \frac{1}{2} \right) = \frac{1}{4}$

Hyperbolic Tangent (tanh) (Deep Learning Specialization, Andrew Ng)

$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

- Shifted version of the sigmoid
- Range: $(-1, +1)$
- Gradient $\rightarrow 0$ for higher values of $|z|$
- Mean around 0
- **Derivative:**



$$\frac{d}{dz} g(z) = g'(z) = 1 - (\tanh(z))^2$$
$$g'(z) = (1 - g(z)^2)$$

$$\text{if } a = g(z), g'(z) = 1 - a^2$$

- Examples:
 - $Z = 10, g(z) \approx 1, g'(z) \approx 0$
 - $Z = -10, g(z) \approx -1, g'(z) \approx 0$
 - $Z = 0, g(z) \approx 0, g'(z) = 1$

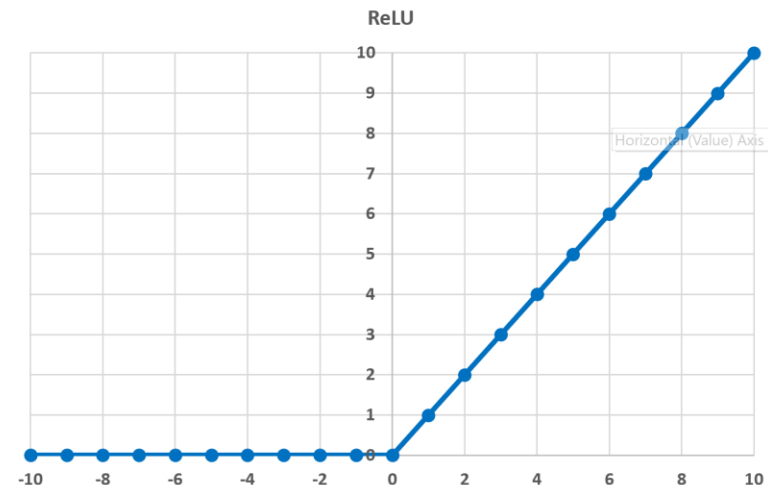
Rectified Linear Unit (ReLU)^(Deep Learning Specialization, Andrew Ng)

$$g(z) = \max(0, z)$$

- Range: $(0, \infty)$
- Solves the problem of diminished gradients for higher values of $|z|$
- Gradient undefined for $z = 0$, and 0 for $z < 0$
- **Derivative:**

$$\frac{d}{dz}g(z) = g'(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z > 0 \\ \text{undef} & \text{if } z = 0 \end{cases}$$

$$g'(z) \approx \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$



Leaky ReLU (Deep Learning Specialization, Andrew Ng)

$$g(z) = \max(0.01z, z)$$

- Range: $(-\infty, \infty)$
- Solves the problem of diminished gradients for higher values of $|z|$
- Gradient undefined for $z = 0$
- **Derivative:**

$$\frac{d}{dz} g(z) = g'(z) = \begin{cases} 0.01 & \text{if } z < 0 \\ 1 & \text{if } z > 0 \\ \text{undef} & \text{if } z = 0 \end{cases}$$

$$g'(z) \approx \begin{cases} 0.01 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$

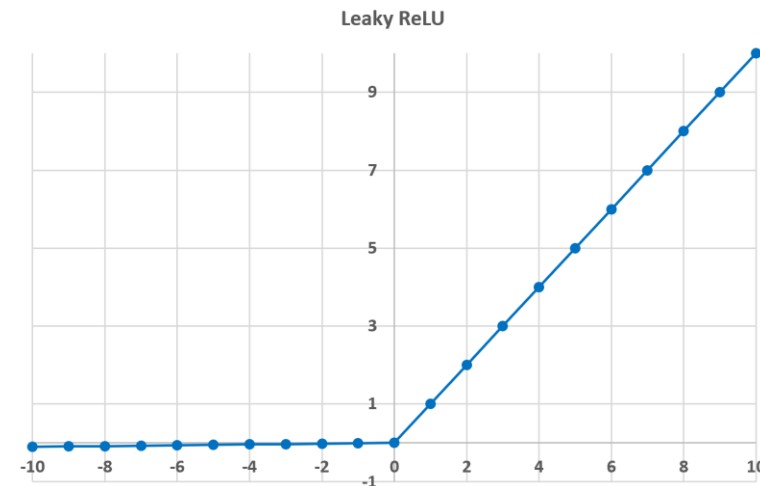


Table of Activation functions

(Activation Functions : Sigmoid, ReLU, Leaky ReLU and Softmax basics for Neural Networks and Deep Learning, Himanshu Sharma, <https://medium.com/@himanshuxd/activation-functions-sigmoid-relu-leaky-relu-and-softmax-basics-for-neural-networks-and-deep-8d9c70eed91e>)

Name	Plot	Equation	Derivative (with respect to x)	Range	Order of continuity	Monotonic	Monotonically derivative	Approximate identity near the origin
Identity		$f(x) = x$	$f'(x) = 1$	$(-\infty, \infty)$	C^∞	Yes	Yes	Yes
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$	$\{0,1\}$	C^{-1}	Yes	No	No
Logistic (a.k.a. Sigmoid or Soft step)		$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$ ^[1]	$f'(x) = f(x)(1 - f(x))$	$(0,1)$	C^∞	Yes	No	No
Tanh		$f(x) = \tanh(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$	$f'(x) = 1 - f(x)^2$	$(-1,1)$	C^∞	Yes	No	Yes
ArcTan		$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2 + 1}$	$(-\frac{\pi}{2}, \frac{\pi}{2})$	C^∞	Yes	No	Yes
ElliotSig ^{[9][10][11]} Softsign ^{[12][13]}		$f(x) = \frac{x}{1 + x }$	$f'(x) = \frac{1}{(1 + x)^2}$	$(-1,1)$	C^1	Yes	No	Yes
Inverse square root unit (ISRU) ^[14]		$f(x) = \frac{x}{\sqrt{1 + \alpha x^2}}$	$f'(x) = \left(\frac{1}{\sqrt{1 + \alpha x^2}} \right)^3$	$(-\frac{1}{\sqrt{\alpha}}, \frac{1}{\sqrt{\alpha}})$	C^∞	Yes	No	Yes
Inverse square root linear unit (ISRLU) ^[14]		$f(x) = \begin{cases} \frac{x}{\sqrt{1 + \alpha x^2}} & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} \left(\frac{1}{\sqrt{1 + \alpha x^2}} \right)^3 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\frac{1}{\sqrt{\alpha}}, \infty)$	C^2	Yes	Yes	Yes
Square Nonlinearity (SQNL) ^[11]		$f(x) = \begin{cases} 1 & : x > 2.0 \\ x - \frac{x^2}{4} & : 0 \leq x \leq 2.0 \\ x + \frac{x^2}{4} & : -2.0 \leq x < 0 \\ -1 & : x < -2.0 \end{cases}$	$f'(x) = 1 \mp \frac{x}{2}$	$(-1,1)$	C^∞	Yes	No	Yes
Rectified linear unit (ReLU) ^[15]		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$[0, \infty)$	C^0	Yes	Yes	No
Bipolar rectified linear unit (BReLU) ^[16]		$f(x_i) = \begin{cases} ReLU(x_i) & \text{if } i \bmod 2 = 0 \\ -ReLU(-x_i) & \text{if } i \bmod 2 \neq 0 \end{cases}$	$f'(x_i) = \begin{cases} ReLU'(x_i) & \text{if } i \bmod 2 = 0 \\ -ReLU'(-x_i) & \text{if } i \bmod 2 \neq 0 \end{cases}$	$(-\infty, \infty)$	C^0	Yes	Yes	No
Leaky rectified linear unit (Leaky ReLU) ^[17]		$f(x) = \begin{cases} 0.01x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0.01 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\infty, \infty)$	C^0	Yes	Yes	No
Parameteric rectified linear unit (PReLU) ^[18]		$f(\alpha, x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(\alpha, x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\infty, \infty)$ ^[21]	C^0	Yes iff $\alpha \geq 0$	Yes	Yes iff $\alpha = 1$
Randomized leaky rectified linear unit (RReLU) ^[19]		$f(\alpha, x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$ ^[21]	$f'(\alpha, x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\infty, \infty)$	C^0	Yes	Yes	No
Exponential linear unit (ELU) ^[20]		$f(\alpha, x) = \begin{cases} \alpha(e^x - 1) & \text{for } x \leq 0 \\ x & \text{for } x > 0 \end{cases}$	$f'(\alpha, x) = \begin{cases} f(\alpha, x) + \alpha & \text{for } x \leq 0 \\ 1 & \text{for } x > 0 \end{cases}$	$(-\alpha, \infty)$	$\begin{cases} C_1^1 & \text{when } \alpha = 1 \\ C_0^1 & \text{otherwise} \end{cases}$	Yes iff $\alpha \geq 0$	Yes iff $0 \leq \alpha \leq 1$	Yes iff $\alpha = 1$
Scaled exponential linear unit (SELU) ^[21]		$f(\alpha, x) = \lambda \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$ with $\lambda = 1.0507$ and $\alpha = 1.67326$	$f'(\alpha, x) = \lambda \begin{cases} \alpha(e^x) & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\lambda\alpha, \infty)$	C^0	Yes	No	No
S-shaped rectified linear activation unit (SReLU) ^[22]		$f_{t_l, a_l, t_r, a_r}(x) = \begin{cases} t_l + a_l(x - t_l) & \text{for } x \leq t_l \\ x & \text{for } t_l < x < t_r \\ t_r + a_r(x - t_r) & \text{for } x \geq t_r \end{cases}$ t_l, a_l, t_r, a_r are parameters.	$f'_{t_l, a_l, t_r, a_r}(x) = \begin{cases} a_l & \text{for } x \leq t_l \\ 1 & \text{for } t_l < x < t_r \\ a_r & \text{for } x \geq t_r \end{cases}$	$(-\infty, \infty)$	C^0	No	No	No
Adaptive piecewise linear (APL) ^[23]		$f(x) = \max(0, x) + \sum_{s=1}^S a_s^* \max(0, -x + b_s^*)$	$f'(x) = H(x) - \sum_{s=1}^S a_s^* H(-x + b_s^*)$ ^[4]	$(-\infty, \infty)$	C^0	No	No	No
SoftPlus ^[24]		$f(x) = \ln(1 + e^x)$	$f'(x) = \frac{1}{1 + e^{-x}}$	$(0, \infty)$	C^∞	Yes	Yes	No
Bent identity		$f(x) = \frac{\sqrt{x^2 + 1} - 1}{2} + x$	$f'(x) = \frac{x}{2\sqrt{x^2 + 1}} + 1$	$(-\infty, \infty)$	C^∞	Yes	Yes	Yes
Sigmoid-weighted linear unit (SiLU) ^[25] (a.k.a. Swish ^[26])		$f(x) = x \cdot \sigma(x)$ ^[30]	$f'(x) = f(x) + \sigma(x)(1 - f(x))$ ^[30]	$[\approx -0.28, \infty)$	C^∞	No	No	No
SoftExponential ^[27]		$f(\alpha, x) = \begin{cases} -\frac{\ln(1 - \alpha(x + \alpha))}{\alpha} & \text{for } \alpha < 0 \\ x & \text{for } \alpha = 0 \\ \frac{e^{\alpha x} - 1}{\alpha} + \alpha & \text{for } \alpha > 0 \end{cases}$	$f'(\alpha, x) = \begin{cases} \frac{1}{1 - \alpha(x + \alpha)} & \text{for } \alpha < 0 \\ e^{\alpha x} & \text{for } \alpha \geq 0 \end{cases}$	$(-\infty, \infty)$	C^∞	Yes	Yes	Yes iff $\alpha = 0$
Soft Clipping ^[28]		$f(\alpha, x) = \frac{1}{\alpha} \log \frac{1 + e^{\alpha x}}{1 + e^{\alpha(x-1)}}$	$f'(\alpha, x) = \frac{1}{2} \sinh\left(\frac{p}{2}\right) \operatorname{sech}\left(\frac{px}{2}\right) \operatorname{sech}\left(\frac{p}{2}(1 - x)\right)$	$(0,1)$	C^∞	Yes	No	No
Sinusoid ^[29]		$f(x) = \sin(x)$	$f'(x) = \cos(x)$	$[-1,1]$	C^∞	No	No	Yes
Sinc		$f(x) = \begin{cases} 1 & \text{for } x = 0 \\ \frac{\sin(x)}{x} & \text{for } x \neq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x = 0 \\ \frac{\cos(x)}{x} - \frac{\sin(x)}{x^2} & \text{for } x \neq 0 \end{cases}$	$[\approx -2.17234, 1]$	C^∞	No	No	No
Gaussian		$f(x) = e^{-x^2}$	$f'(x) = -2xe^{-x^2}$	$(0,1]$	C^∞	No	No	No

Up next

- Forward and Backward passes
 - Backpropagation
 - Logistic Regression
 - NN

Neural Networks

Training the NN – Backpropagation

Agha Ali Raza



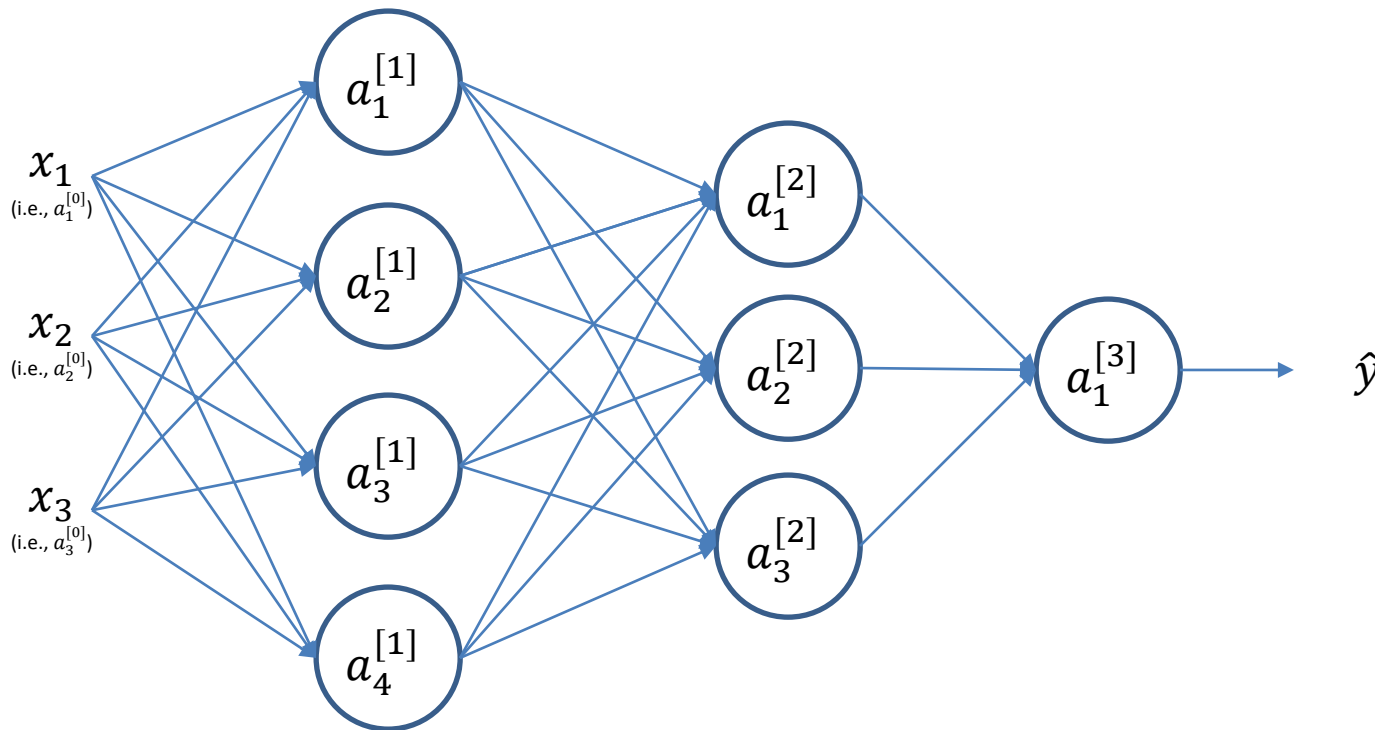
Sources

- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell, Video Lectures 35-37,
https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecture_note20.html
- **Deep Learning Specialization**, Andrew Ng
https://www.coursera.org/specializations/deep-learning?utm_source=deeplearningai&utm_medium=institutions&utm_campaign=SocialYoutubeDLSC1W1L1
 - Video Lectures: C1W3L1 – C1W4L6:
<https://www.youtube.com/playlist?list=PLpFsSf5Dm-pd5d3rjNtIXUHT-v7bdaEle>
- **A beginner's guide to deriving and implementing backpropagation**, by Pranav Budhwant <https://link.medium.com/Zp3zxNWpf6>
- **Tensorflow playground**: <https://playground.tensorflow.org/>
- **Machine Learning Playground**: <https://ml-playground.com/#>

How to train your NN?

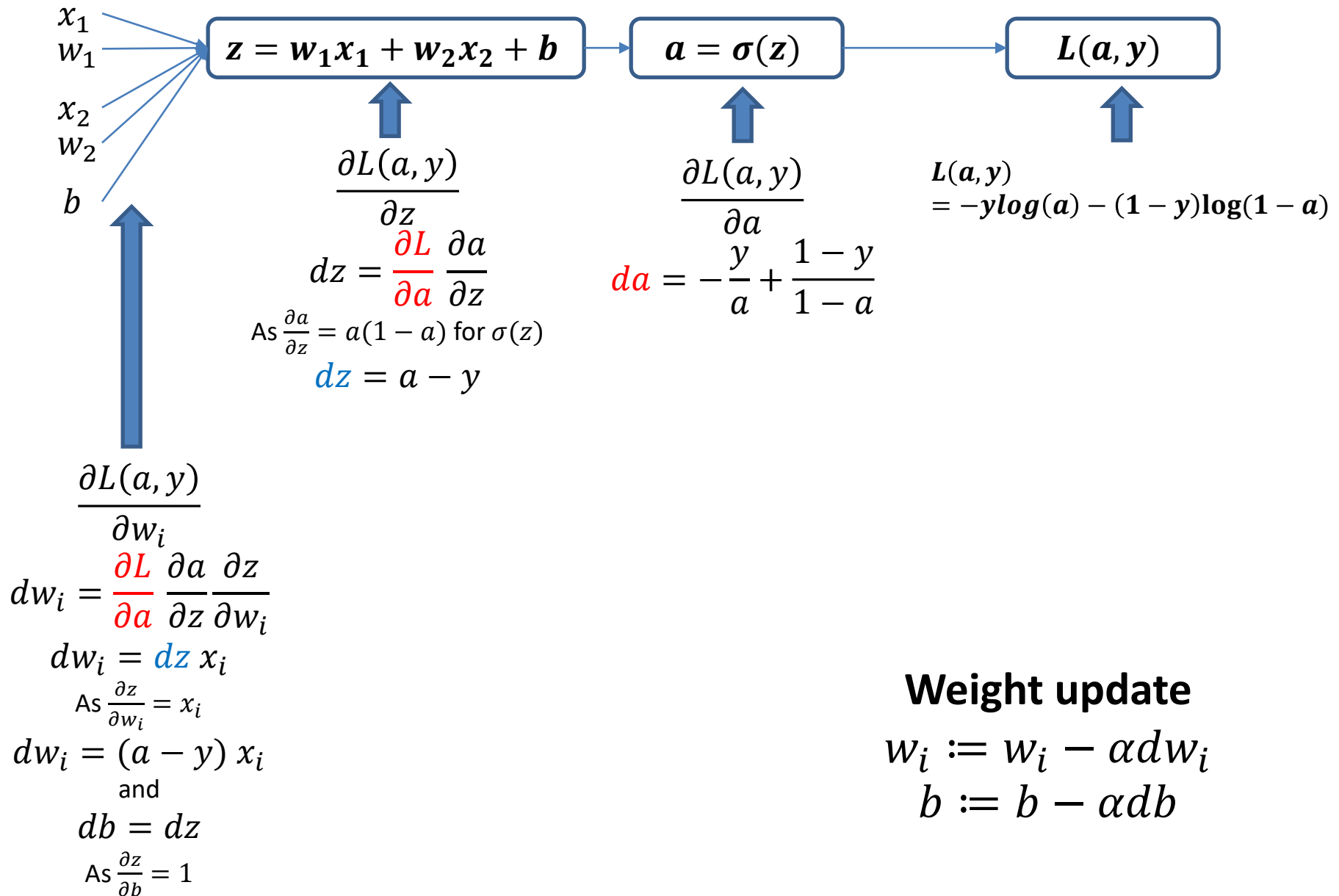
Consider the following 2-layer NN:

- Number of nodes in layer l : $n^{[l]}$
- $a_i^{[l]}$: i th node in layer l
- $a^{[0]} = X$
- $a^{[L]} = h(x) = \hat{y}$



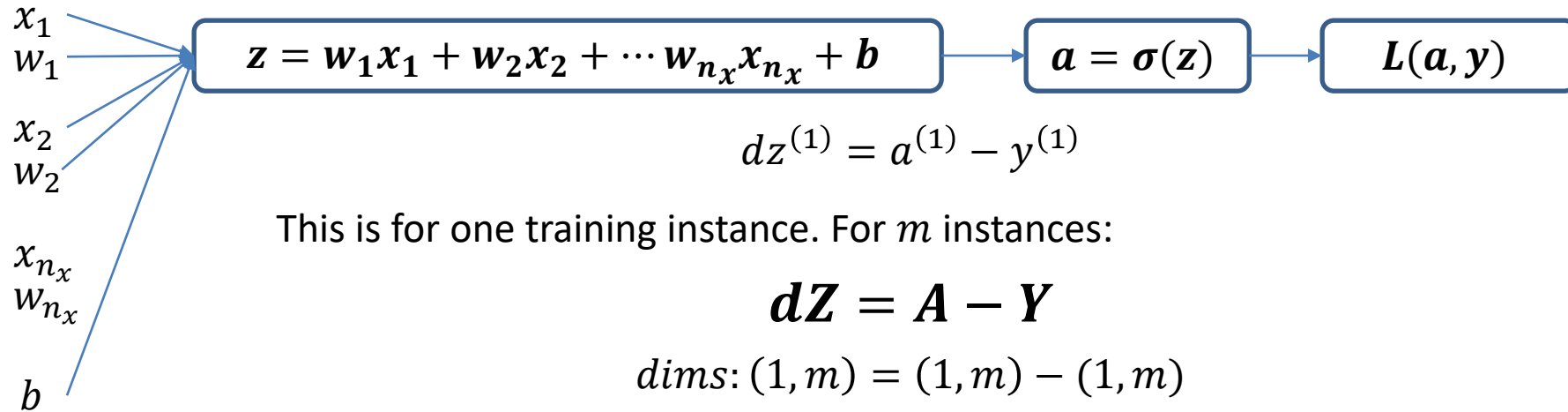
Logistic Regression Derivatives w.r.t Cost L

(Deep Learning Specialization, Andrew Ng)



Vectorizing Logistic Regression – The Backward Pass

(Deep Learning Specialization, Andrew Ng)



This is for one training instance. For m instances:

$$dZ = A - Y$$

$$\text{dims: } (1, m) = (1, m) - (1, m)$$

$$db = \frac{1}{m} \sum_{i=1}^m dz^{(i)} \Rightarrow$$

$$db = \frac{1}{m} \text{sum}(dZ), \text{dims} = (1, 1)$$

$$dW = \frac{1}{m} X dZ^T, \text{dims: } (n_x, 1) = (n_x, m)(m, 1)$$

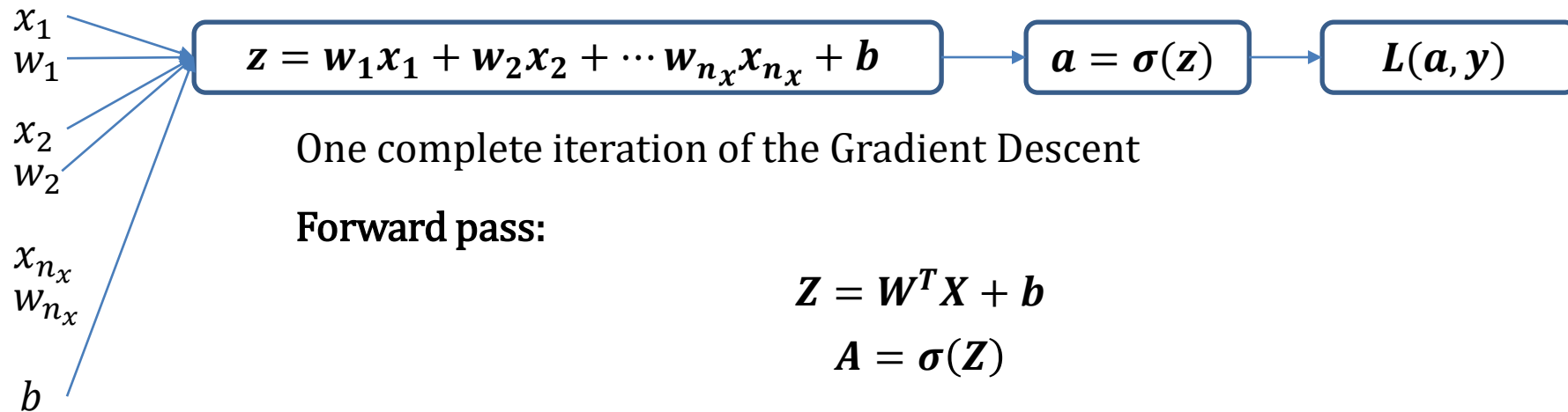
$$W := W - \alpha(dW)$$

$$b := b - \alpha(db)$$

- $\frac{\partial a}{\partial z} = a(1 - a)$ for $\sigma(z)$
- $\frac{\partial z}{\partial w_i} = x_i$
- $da = -\frac{y}{a} + \frac{1-y}{1-a}$
- $dz = a - y$
- $dw_i = dz x_i$
- $dw_i = (a - y) x_i$
- $db = dz$

Vectorizing Logistic Regression – The Backward Pass

(Deep Learning Specialization, Andrew Ng)



One complete iteration of the Gradient Descent

Forward pass:

$$Z = W^T X + b$$

$$A = \sigma(Z)$$

Backward pass:

$$dZ = A - Y$$

$$dW = \frac{1}{m} X dZ^T$$

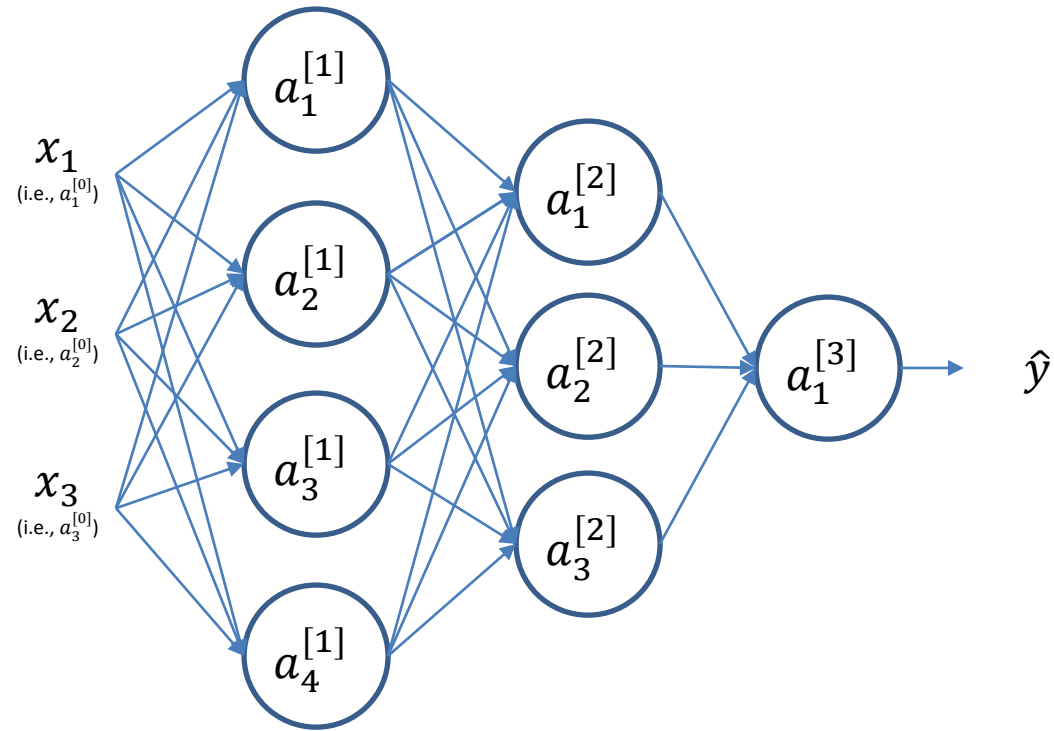
$$db = \frac{1}{m} \text{sum}(dZ)$$

$$W := W - \alpha(dw)$$

$$b := b - \alpha(db)$$

- $\frac{\partial a}{\partial z} = a(1 - a)$ for $\sigma(z)$
- $\frac{\partial z}{\partial w_i} = x_i$
- $da = -\frac{y}{a} + \frac{1-y}{1-a}$
- $dz = a - y$
- $dw_i = dz x_i$
- $dw_i = (a - y) x_i$
- $db = dz$

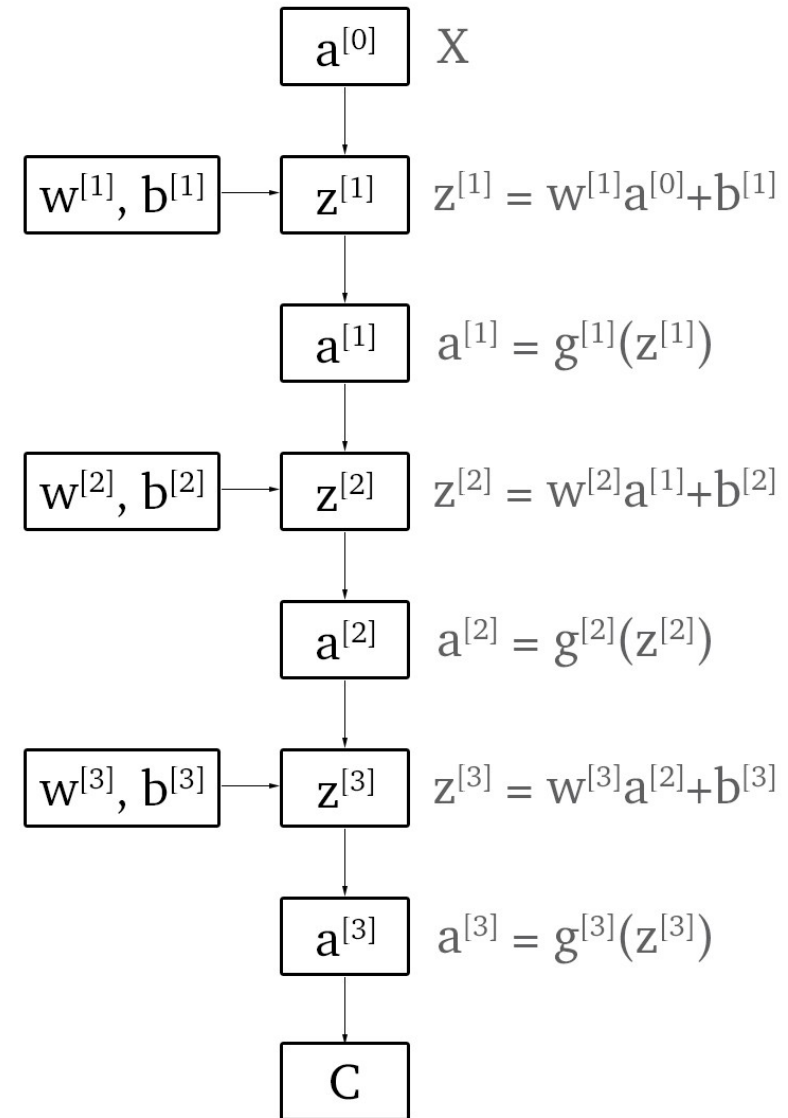
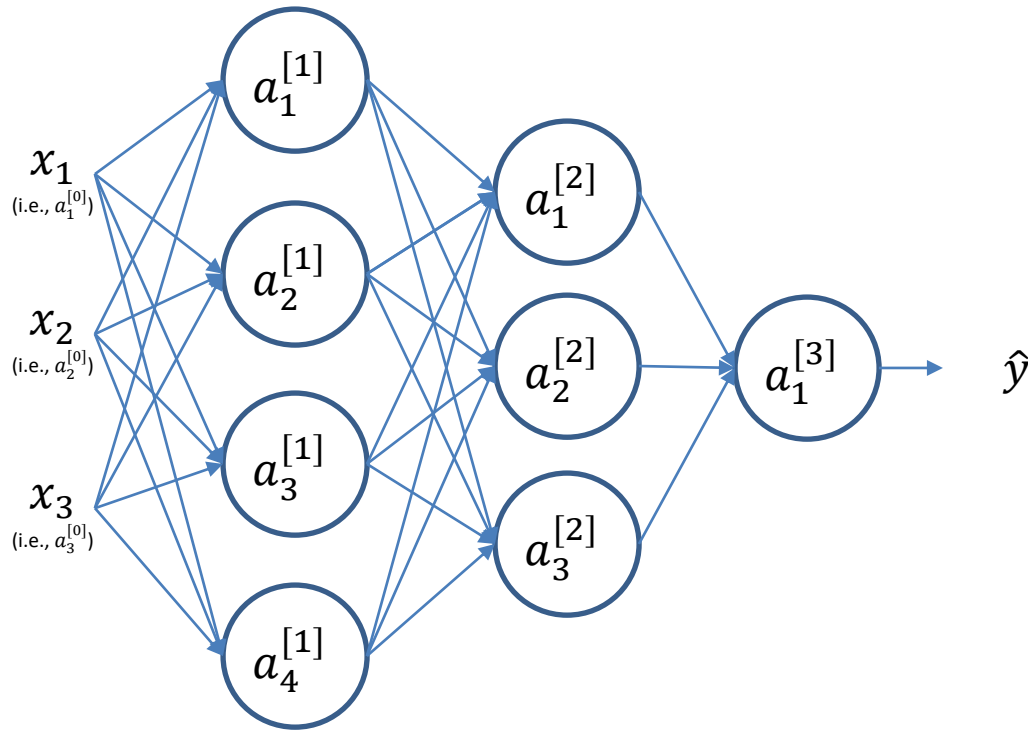
Now the NN?



The forward pass

(A beginner's guide to deriving and implementing backpropagation)

Pranav Budhwant, <https://medium.com/binaryandmore/beginners-guide-to-deriving-and-implementing-backpropagation-e3c1a5a1e536>)

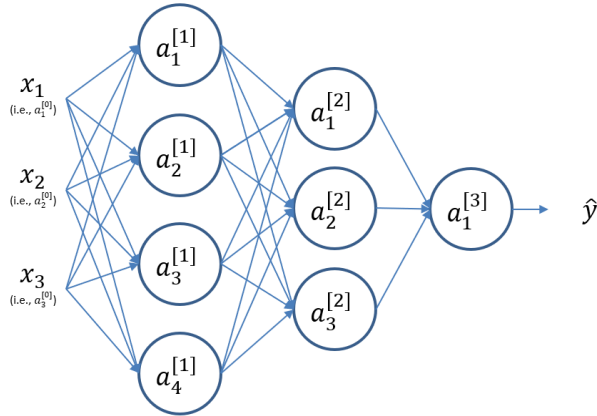


$$C = -(y \log a^{[3]} + (1 - y) \log (1 - a^{[3]}))$$

The backward pass

(A beginner's guide to deriving and implementing backpropagation)

Pranav Budhwant, <https://medium.com/binaryandmore/beginners-guide-to-deriving-and-implementing-backpropagation-e3c1a5a1e536>



$$w^{[l]} = w^{[l]} - \alpha \frac{\partial C}{\partial w^{[l]}}$$

$$b^{[l]} = b^{[l]} - \alpha \frac{\partial C}{\partial b^{[l]}}$$

$$\frac{\partial C}{\partial w^{[3]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial w^{[3]}}$$

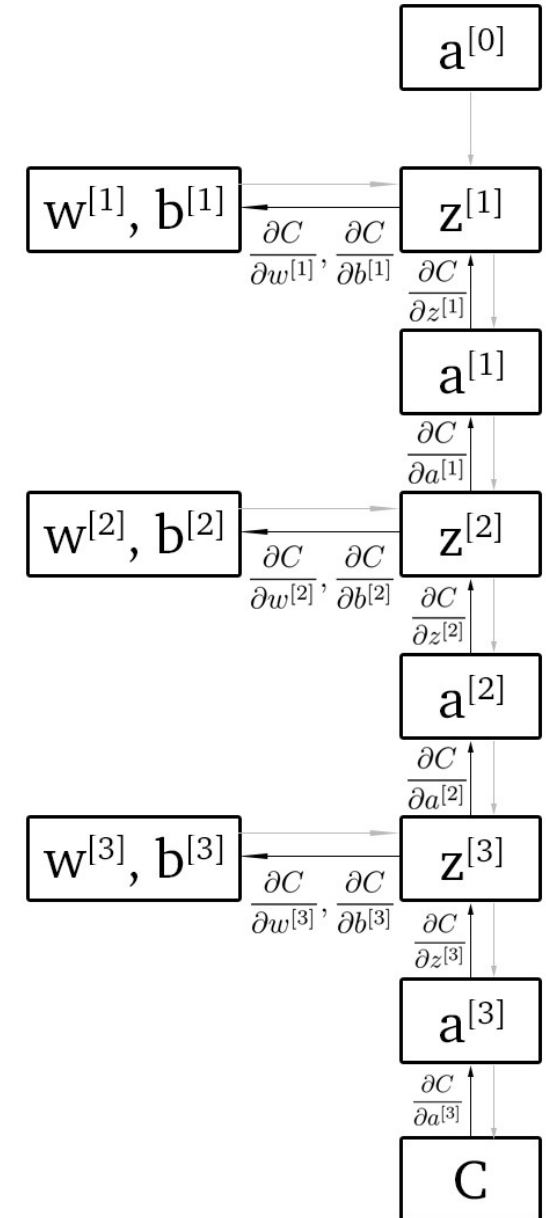
$$\frac{\partial C}{\partial b^{[3]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial b^{[3]}}$$

$$\frac{\partial C}{\partial w^{[2]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial w^{[2]}}$$

$$\frac{\partial C}{\partial b^{[2]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial b^{[2]}}$$

$$\frac{\partial C}{\partial w^{[1]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial z^{[1]}} \cdot \frac{\partial z^{[1]}}{\partial w^{[1]}}$$

$$\frac{\partial C}{\partial b^{[1]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial z^{[1]}} \cdot \frac{\partial z^{[1]}}{\partial b^{[1]}}$$



The backward pass (Layer L)

(A beginner's guide to deriving and implementing backpropagation)

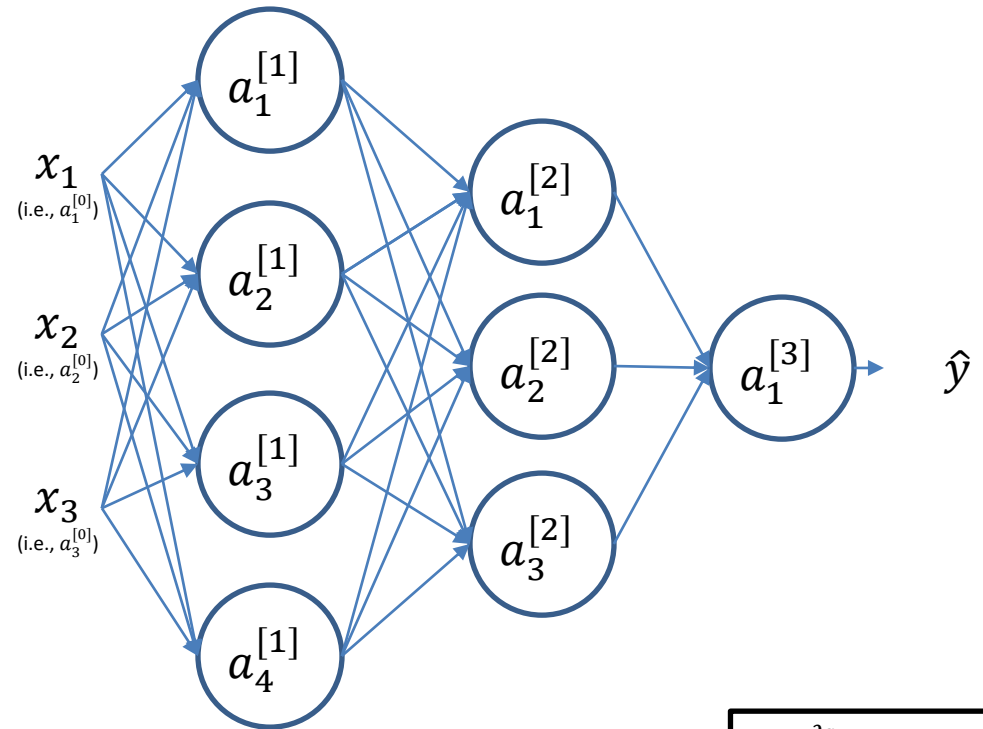
Pranav Budhwant, <https://medium.com/binaryandmore/beginners-guide-to-deriving-and-implementing-backpropagation-e3c1a5a1e536>

$$\frac{\partial C}{\partial w^{[3]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial w^{[3]}}$$

$$\frac{\partial C}{\partial b^{[3]}} = \underbrace{\frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}}}_{\delta^{[3]}} \cdot \underbrace{\frac{\partial z^{[3]}}{\partial b^{[3]}}}_{1}$$

$$\frac{\partial C}{\partial w^{[l]}} = \frac{\partial C}{\partial z^{[l]}} \cdot \frac{\partial z^{[l]}}{\partial w^{[l]}}$$

$$\frac{\partial C}{\partial b^{[l]}} = \frac{\partial C}{\partial z^{[l]}} \cdot \frac{\partial z^{[l]}}{\partial b^{[l]}}$$



$$\frac{\partial C}{\partial z^{[L]}} = a^{[L]} - y$$

$$\frac{\partial z^{[l]}}{\partial b^{[l]}} = 1$$

$$\frac{\partial z^{[l]}}{\partial w^{[l]}} = a^{[l-1]}$$

$$\frac{\partial C}{\partial w^{[l]}} = \frac{\partial C}{\partial z^{[l]}} \cdot a^{[l-1]^T}$$

$$\frac{\partial C}{\partial b^{[l]}} = \frac{\partial C}{\partial z^{[l]}}$$

- $\frac{\partial a}{\partial z} = a(1 - a)$
for $\sigma(z)$
- $\frac{\partial z}{\partial w_i} = x_i$
- $\frac{\partial z}{\partial b} = 1$
- $da = -\frac{y}{a} + \frac{1-y}{1-a}$
- $dz = a - y$
- $dw_i = dz x_i$
- $dw_i = (a - y) x_i$
- $db = dz$

The backward pass (Notes)

(A beginner's guide to deriving and implementing backpropagation)

Pranav Budhwant, <https://medium.com/binaryandmore/beginners-guide-to-deriving-and-implementing-backpropagation-e3c1a5a1e536>

$$\begin{aligned}\frac{\partial C}{\partial z^{[L]}} &= -\left(\frac{y}{a^{[L]}} - \frac{1-y}{1-a^{[L]}}\right) \cdot a^{[L]} \cdot (1-a^{[L]}) \\&= -\left(\frac{y(1-a^{[L]}) - (1-y)a^{[L]}}{a^{[L]} \cdot (1-a^{[L]})}\right) \cdot a^{[L]} \cdot (1-a^{[L]}) \\&= -(y(1-a^{[L]}) - (1-y)a^{[L]}) \\&= -(y - ya^{[L]} - (a^{[L]} - ya^{[L]})) \\&= -(y - ya^{[L]} - a^{[L]} + ya^{[L]}) \\&= -(y - a^{[L]}) \\&= a^{[L]} - y\end{aligned}$$

$$\therefore \boxed{\frac{\partial C}{\partial z^{[L]}} = a^{[L]} - y}$$

$$\begin{aligned}z^{[l]} &= w^{[l]} \cdot a^{[l-1]} + b^{[l]} \\ \frac{\partial z^{[l]}}{\partial w^{[l]}} &= \frac{\partial}{\partial w^{[l]}} (w^{[l]} \cdot a^{[l-1]} + b^{[l]}) \\ &= a^{[l-1]}\end{aligned}$$

$$\therefore \boxed{\frac{\partial z^{[l]}}{\partial w^{[l]}} = a^{[l-1]}}$$

$$\begin{aligned}z^{[l]} &= w^{[l]} \cdot a^{[l-1]} + b^{[l]} \\ \frac{\partial z^{[l]}}{\partial b^{[l]}} &= \frac{\partial}{\partial b^{[l]}} (w^{[l]} \cdot a^{[l-1]} + b^{[l]})\end{aligned}$$

$$\therefore \boxed{\frac{\partial z^{[l]}}{\partial b^{[l]}} = 1}$$

The backward pass (Layer $L - 1$ and beyond)

(A beginner's guide to deriving and implementing backpropagation)

Pranav Budhwant, <https://medium.com/binaryandmore/beginners-guide-to-deriving-and-implementing-backpropagation-e3c1a5a1e536>

$$\frac{\partial C}{\partial w^{[2]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial w^{[2]}}$$

$$\frac{\partial C}{\partial b^{[2]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial b^{[2]}}$$

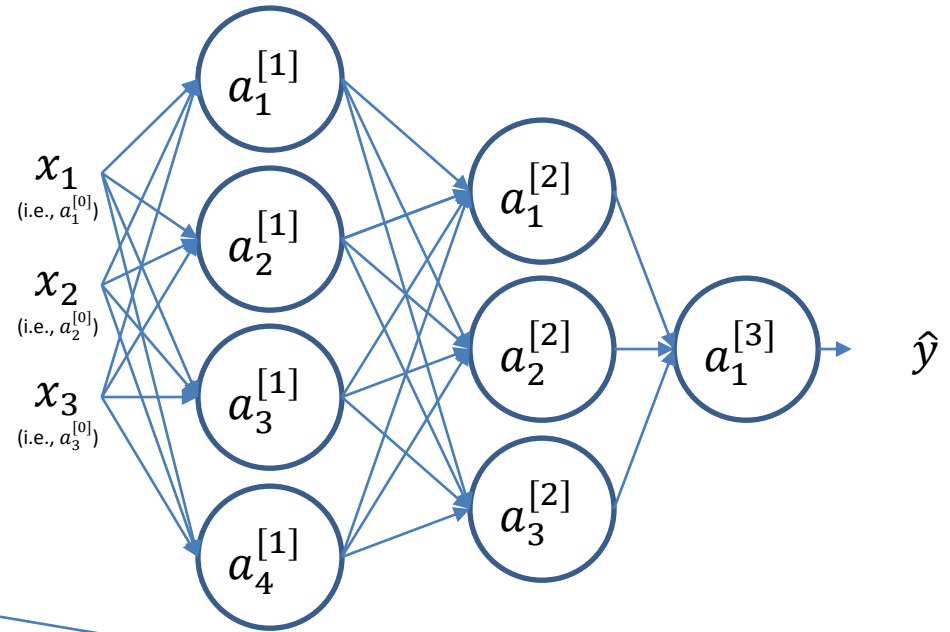
$$\frac{\partial C}{\partial z^{[l]}} = \frac{\partial C}{\partial z^{[l+1]}} \cdot \frac{\partial z^{[l+1]}}{\partial a^{[l]}} \cdot \frac{\partial a^{[l]}}{\partial z^{[l]}}$$

$$\frac{\partial C}{\partial w^{[l]}} = \frac{\partial C}{\partial z^{[l]}} \cdot \frac{\partial z^{[l]}}{\partial w^{[l]}}$$

$$\frac{\partial C}{\partial b^{[l]}} = \frac{\partial C}{\partial z^{[l]}} \cdot \frac{\partial z^{[l]}}{\partial b^{[l]}}$$

$$\frac{\partial z^{[l]}}{\partial b^{[l]}} = 1$$

$$\frac{\partial z^{[l]}}{\partial w^{[l]}} = a^{[l-1]}$$



$$\frac{\partial C}{\partial z^{[l]}} = (w^{[l+1]^T} \cdot \frac{\partial C}{\partial z^{[l+1]}}) \cdot \sigma'(z^{[l]})$$

Recursive structure that allows us to incorporate $\frac{\partial C}{\partial z^{l+1}}$ that becomes $a^{[L]} - y$ for $\frac{\partial C}{\partial z^L}$

$$\frac{\partial C}{\partial w^{[l]}} = \frac{\partial C}{\partial z^{[l]}} \cdot a^{[l-1]^T}$$

$$\frac{\partial C}{\partial b^{[l]}} = \frac{\partial C}{\partial z^{[l]}}$$

- $\frac{\partial a}{\partial z} = a(1 - a)$ for $\sigma(z)$
- $\frac{\partial z}{\partial w_i} = x_i$
- $da = -\frac{y}{a} + \frac{1-y}{1-a}$
- $dz = a - y$
- $dw_i = dz x_i$
- $dw_i = (a - y) x_i$
- $db = dz$

The backward pass (Layer $L - 1$ and beyond)

(A beginner's guide to deriving and implementing backpropagation)

Pranav Budhwant, <https://medium.com/binaryandmore/beginners-guide-to-deriving-and-implementing-backpropagation-e3c1a5a1e536>

$$\frac{\partial C}{\partial w^{[2]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial w^{[2]}}$$

$$\frac{\partial C}{\partial b^{[2]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial b^{[2]}}$$

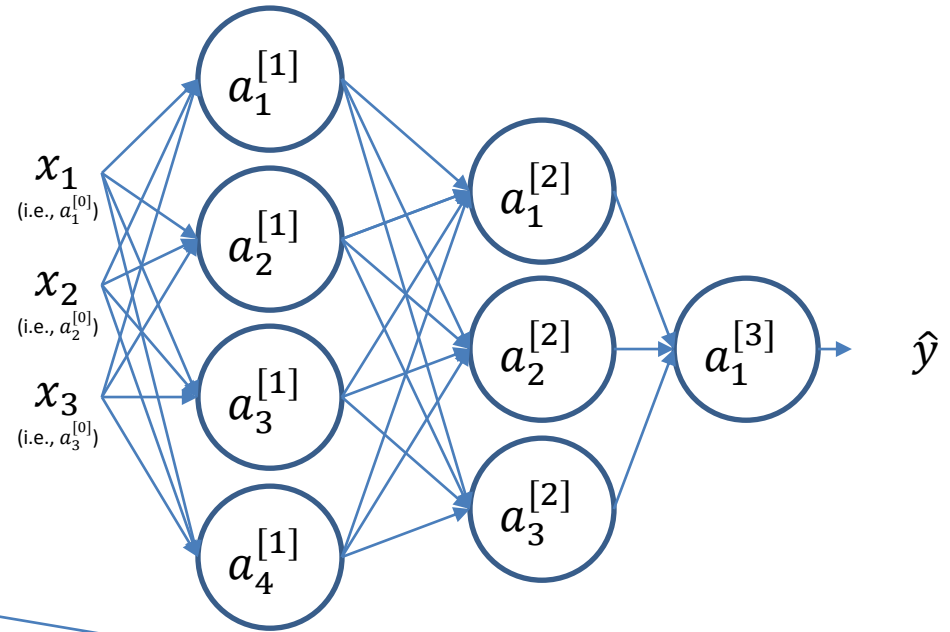
$$\frac{\partial C}{\partial z^{[l]}} = \frac{\partial C}{\partial z^{[l+1]}} \cdot \frac{\partial z^{[l+1]}}{\partial a^{[l]}} \cdot \frac{\partial a^{[l]}}{\partial z^{[l]}}$$

$$\frac{\partial C}{\partial w^{[l]}} = \frac{\partial C}{\partial z^{[l]}} \cdot \frac{\partial z^{[l]}}{\partial w^{[l]}}$$

$$\frac{\partial C}{\partial b^{[l]}} = \frac{\partial C}{\partial z^{[l]}} \cdot \frac{\partial z^{[l]}}{\partial b^{[l]}}$$

$$\frac{\partial z^{[l]}}{\partial b^{[l]}} = 1$$

$$\frac{\partial z^{[l]}}{\partial w^{[l]}} = a^{[l-1]}$$



$$\frac{\partial C}{\partial z^{[l]}} = \frac{\partial C}{\partial z^{[l+1]}} \cdot \frac{\partial z^{[l+1]}}{\partial a^{[l]}} \cdot \frac{\partial a^{[l]}}{\partial z^{[l]}}$$

$$\frac{\partial C}{\partial z^{[l]}} = (w^{[l+1]^T} \cdot \frac{\partial C}{\partial z^{[l+1]}}) \cdot \sigma'(z^{[l]})$$

Recursive structure that allows us to incorporate $\frac{\partial C}{\partial z^{[l+1]}}$ that becomes $a^{[L]} - y$ for $\frac{\partial C}{\partial z^L}$

$$\frac{\partial C}{\partial w^{[l]}} = \frac{\partial C}{\partial z^{[l]}} \cdot a^{[l-1]^T}$$

$$\frac{\partial C}{\partial b^{[l]}} = \frac{\partial C}{\partial z^{[l]}}$$

- $\frac{\partial a}{\partial z} = a(1 - a)$ for $\sigma(z)$
- $\frac{\partial z}{\partial w_i} = x_i$
- $da = -\frac{y}{a} + \frac{1-y}{1-a}$
- $dz = a - y$
- $dw_i = dz x_i$
- $dw_i = (a - y) x_i$
- $db = dz$

The backward pass (Notes)

(A beginner's guide to deriving and implementing backpropagation)

Pranav Budhwant, <https://medium.com/binaryandmore/beginners-guide-to-deriving-and-implementing-backpropagation-e3c1a5a1e536>

$$\frac{\partial C}{\partial z^{[l]}} = \frac{\partial C}{\partial z^{[l+1]}} \cdot \frac{\partial z^{[l+1]}}{\partial a^{[l]}} \cdot \frac{\partial a^{[l]}}{\partial z^{[l]}}$$

$$z^{[l+1]} = w^{[l+1]} \cdot a^{[l]} + b^{[l+1]}$$

$$\frac{\partial z^{[l+1]}}{\partial a^{[l]}} = \frac{\partial}{\partial a^{[l]}} (w^{[l+1]} \cdot a^{[l]} + b^{[l+1]})$$

$$= w^{[l+1]}$$

$$\therefore \frac{\partial z^{[l+1]}}{\partial a^{[l]}} = w^{[l+1]}$$

also,

$$\frac{\partial a^{[l]}}{\partial z^{[l]}} = g^{[l]'}(z^{[l]})$$

$$\frac{\partial a^{[l]}}{\partial z^{[l]}} = \sigma'(z^{[l]})$$

$$\frac{\partial C}{\partial z^{[l]}} = (w^{[l+1]}{}^T \cdot \frac{\partial C}{\partial z^{[l+1]}}) \cdot \sigma'(z^{[l]})$$

$$\sigma(z^{[L]}) = \frac{1}{1 + e^{-z^{[L]}}}$$

$$\sigma'(z^{[L]}) = \frac{d}{dz^{[L]}} \left(\frac{1}{1 + e^{-z^{[L]}}} \right)$$

$$= -\frac{1}{(1 + e^{-z^{[L]}})^2} \cdot \frac{d}{dz^{[L]}} (1 + e^{-z^{[L]}})$$

$$= -\frac{e^{-z^{[L]}}}{(1 + e^{-z^{[L]}})^2} \cdot \frac{d}{dz^{[L]}} (-z^{[L]})$$

$$= \frac{e^{-z^{[L]}}}{(1 + e^{-z^{[L]}})^2}$$

$$= \frac{e^{-z^{[L]}} + 1 - 1}{(1 + e^{-z^{[L]}})^2}$$

$$= \frac{1}{1 + e^{-z^{[L]}}} \cdot \left(\frac{e^{-z^{[L]}} + 1 - 1}{1 + e^{-z^{[L]}}} \right)$$

$$= \frac{1}{1 + e^{-z^{[L]}}} \cdot \left(\frac{1 + e^{-z^{[L]}}}{1 + e^{-z^{[L]}}} - \frac{1}{1 + e^{-z^{[L]}}} \right)$$

$$= \frac{1}{1 + e^{-z^{[L]}}} \cdot \left(1 - \frac{1}{1 + e^{-z^{[L]}}} \right)$$

$$= \sigma(z^{[L]}) \cdot (1 - \sigma(z^{[L]}))$$

$$\sigma'(z^{[L]}) = \sigma(z^{[L]}) \cdot (1 - \sigma(z^{[L]}))$$

$$\therefore \sigma'(z^{[L]}) = a^{[L]} \cdot (1 - a^{[L]}) \quad \dots 2$$

The Forward and Backward Pass

(Deep Learning Specialization, Andrew Ng)

One Iteration of the forward and Backward passes for a 2-layered NN:

Forward Propagation:

$$\begin{aligned} Z^{[1]} &= W^{[1]}X + b^{[1]} & (n^{[1]}, m) &= (n^{[1]}, n^{[0]})(n^{[0]}, m) + (n^{[1]}, m) \\ A^{[1]} &= g^{[1]}(Z^{[1]}) & (n^{[1]}, m) &= (n^{[1]}, m) \\ Z^{[2]} &= W^{[2]}A^{[1]} + b^{[2]} & (n^{[2]}, m) &= (n^{[2]}, n^{[1]})(n^{[1]}, m) + (n^{[2]}, m) \\ A^{[2]} &= g^{[2]}(Z^{[2]}) & (n^{[2]}, m) &= (n^{[2]}, m) \end{aligned}$$

Back Propagation:

$$\begin{aligned} dZ^{[2]} &= A^{[2]} - Y & (n^{[2]}, m) &= (n^{[2]}, m) - (n^{[2]}, m), n^{[2]} = \\ & \text{1 if output layer has 1 node} \\ dW^{[2]} &= \frac{1}{m} dZ^{[2]} A^{[1]T} & (n^{[2]}, n^{[1]}) &= (n^{[2]}, m)(m, n^{[1]}) \\ db^{[2]} &= \frac{1}{m} \text{sum}(dZ^{[2]}) & (n^{[2]}, m) &= (n^{[2]}, m), \text{just replicate the sum } m \text{ times} \end{aligned}$$

$$\begin{aligned} dZ^{[1]} &= W^{[2]T} dZ^{[2]} * g^{[1]'}(Z^{[1]}) & (n^{[1]}, m) &= (n^{[1]}, n^{[2]})(n^{[2]}, m) * (n^{[1]}, m) \\ dW^{[1]} &= \frac{1}{m} dZ^{[1]} X^T & (n^{[1]}, n^{[0]}) &= (n^{[1]}, m)(m, n^{[0]}) \\ db^{[1]} &= \frac{1}{m} \text{sum}(dZ^{[1]}) & (n^{[1]}, m) &= (n^{[1]}, m) \end{aligned}$$

Update:

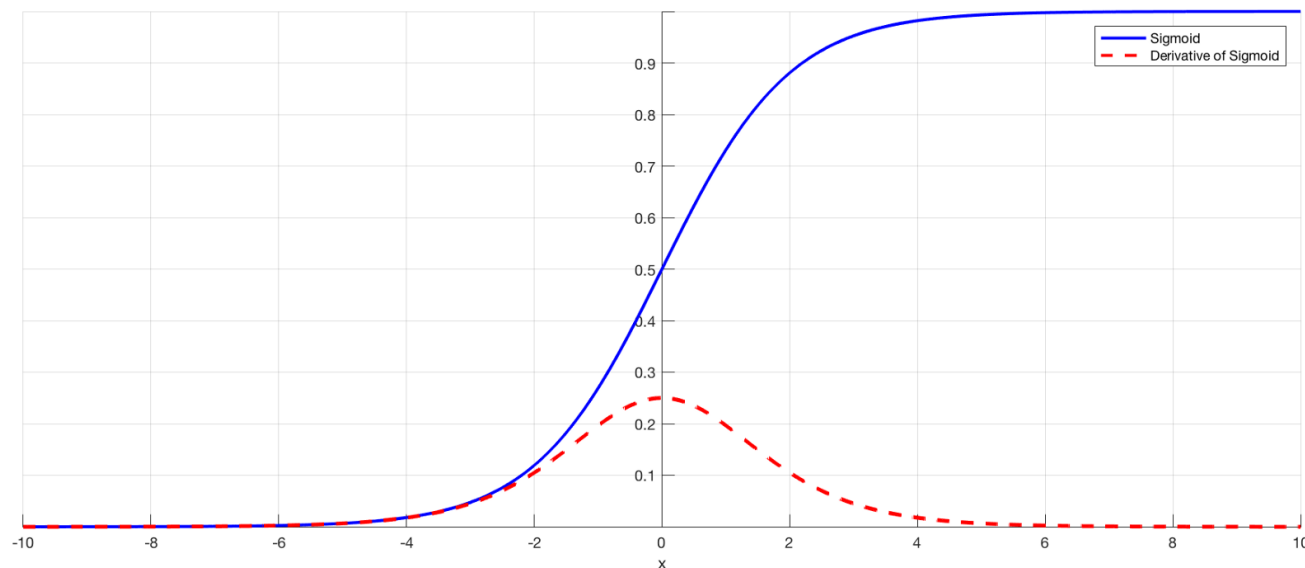
$$\begin{aligned} W^{[2]} &= W^{[2]} - \alpha dW^{[2]} \\ b^{[2]} &= b^{[2]} - \alpha db^{[2]} \end{aligned}$$

$$\begin{aligned} W^{[1]} &= W^{[1]} - \alpha dW^{[1]} \\ b^{[1]} &= b^{[1]} - \alpha db^{[1]} \end{aligned}$$

The Vanishing Gradients Problem

(<https://towardsdatascience.com/the-vanishing-gradient-problem-69bf08b15484>)

- **The problem:** As more layers using certain activation functions are added to neural networks, the gradients of the loss function approaches zero, making the network hard to train.
- **The Reason:** Activation functions like the sigmoid and tanh, squish a large input space into a small input space between 0 and 1.
 - A large change in the input of the sigmoid function will cause a small change in the output. Hence, the derivative becomes small.
 - The derivatives are small for large values of the input $|x|$



The Vanishing Gradients Problem

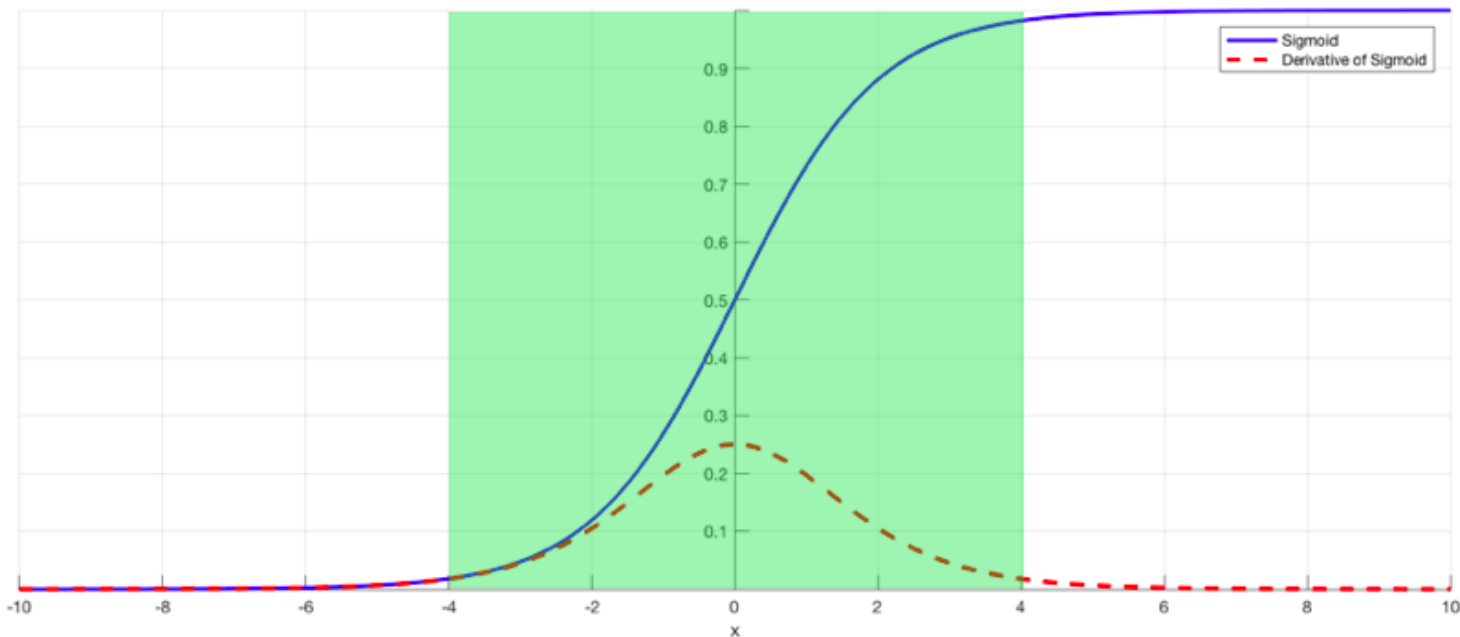
(<https://towardsdatascience.com/the-vanishing-gradient-problem-69bf08b15484>)

- For shallow networks with only a few layers that use these activations, this isn't a big problem.
- However, when more layers are used, it can cause the gradient to be too small for training to work effectively.
- Gradients of neural networks are found using backpropagation.
 - Backpropagation finds the derivatives of the network by moving layer by layer from the final layer to the initial one.
 - By the chain rule, the derivatives of each layer are multiplied down the network (from the final layer to the initial) to compute the derivatives of the initial layers.
 - When n hidden layers use an activation like the sigmoid function, n small derivatives are multiplied together. Thus, the gradient decreases exponentially as we propagate down to the initial layers.
- **A small gradient means that the weights and biases of the initial layers will not be updated effectively with each training session. Since these initial layers are often crucial to recognizing the core elements of the input data, it can lead to overall inaccuracy of the whole network.**

The Vanishing Gradients Problem

(<https://towardsdatascience.com/the-vanishing-gradient-problem-69bf08b15484>)

- The simplest solution is to use other activation functions, such as ReLU, which do not cause small derivatives.
- Another solution is batch normalization i.e. normalize the input so $|x|$ doesn't reach the outer edges of the sigmoid function.
 - Normalize the input so that most of it falls in the green region, where the derivative isn't too small.



For more details please visit
<http://aghaaliraza.com>



Thank you!